

Proyecto Global Integrador: Control Semi-Automático Coordinado de Grúa Portuaria de Muelle Tipo Pórtico

Rodrigo Pérez, Tomás Suárez

13 de septiembre de 2026

Resumen

Con este trabajo se sintetizan los conocimientos adquiridos en la materia *Autómatas y Control Discreto* a partir de la implementación, de manera simplificada, del control semi-automático de una grúa portuaria de contenedores de tipo pórtico.

Tres niveles de control son diseñados e implementados en el mismo: nivel 0 de seguridad, nivel 1 supervisor y nivel 2 regulador. Los dos primeros se desarrollan como máquinas de estados finitos; mientras que el nivel 2 se desarrolla como estados continuos en tiempo discreto, el cual se basa en el control *feedforward* no lineal con *feedback* PD, lo cual permite un correcto seguimiento de consigna en conjunto con una amortiguación del ángulo de balanceo de la carga.

En una primera etapa se diseñan los niveles 0 y 1 en stateflow, mientras que el nivel 2 y la planta se diseñan en Simulink. Y se verifica el funcionamiento de todos los sistemas en conjunto.

Y en una segunda etapa, se implementan los niveles 0 y 1 en CODESYS, comunicando el PLC virtual de este entorno con Simulink mediante una arquitectura servidor/cliente basada en el estándar OPC UA, mientras que el nivel 2 se lo implementa en funciones de MATLAB similar al texto estructurado (ST), en tiempo discreto.

1 INTRODUCCIÓN

El presente proyecto es la resolución del trabajo [1] presentado por la cátedra. A lo largo del mismo se hace referencia a ecuaciones e imágenes presentes en el.

La resolución del trabajo procede en dos etapas, de la siguiente manera:

En la primera etapa, se utiliza el enfoque *Model-in-the-loop* en donde se comienza con el desarrollo de la **planta** en Simulink y su **visualización**. Posteriormente, se readaptan las ecuaciones dinámicas del sistema proporcionadas en [1] a fin de obtener 3 ecuaciones dinámicas que representen los comportamientos del sistema de carro, de izaje y de ángulo de balanceo, con las cuales se diseña el **control regulador**. El cuál consiste en el control de dichos 3 subsistemas mediante el control de tipo *feedforward* no lineal con *feedback* PD, tal cómo se cita en [2], y se lo implementa en Simulink.

A continuación, se procede obtener la generación de consignas de movimiento para el mando manual; y luego la generación de la trayectoria para el modo automático implementada en stateflow en el **control supervisor**, junto con la estimación automática de la carga de los contenedores y el manejo automático de los frenos de operación. De manera similar, se desarrolla el **control de seguridad** en stateflow el cual incluye al manejo automático de los frenos de emergencia, y la implementación del watchdog. Y se verifica el correcto funcionamiento de todos los sistemas en conjunto.

En la segunda etapa, se utiliza el enfoque *Software-in-the-loop* en donde se implementan los niveles 0 y 1 en CODESYS, utilizando lenguajes de programación bajo norma IEC 61131-3, principalmente Sequential Function Chart (SFC) y Structured Text (ST). Mientras que el nivel 2 se lo implementa en funciones de MATLAB similar a ST, en tiempo discreto.

Luego, se realiza la conexión entre estos dos entornos lanzando un servidor OPC UA [3] en el PLC virtual de CODESYS, accediendo a este mediante un cliente OPC UA en Simulink [4].

Finalmente se presentan conclusiones y propuestas de mejora.

2 DESARROLLO

2.1 PRIMERA ETAPA, STATEFLOW Y SIMULINK

2.1.1 PLANTA

A continuación, en base a las ecuaciones presentadas en [1], se implementa la dinámica de la grúa en Simulink.

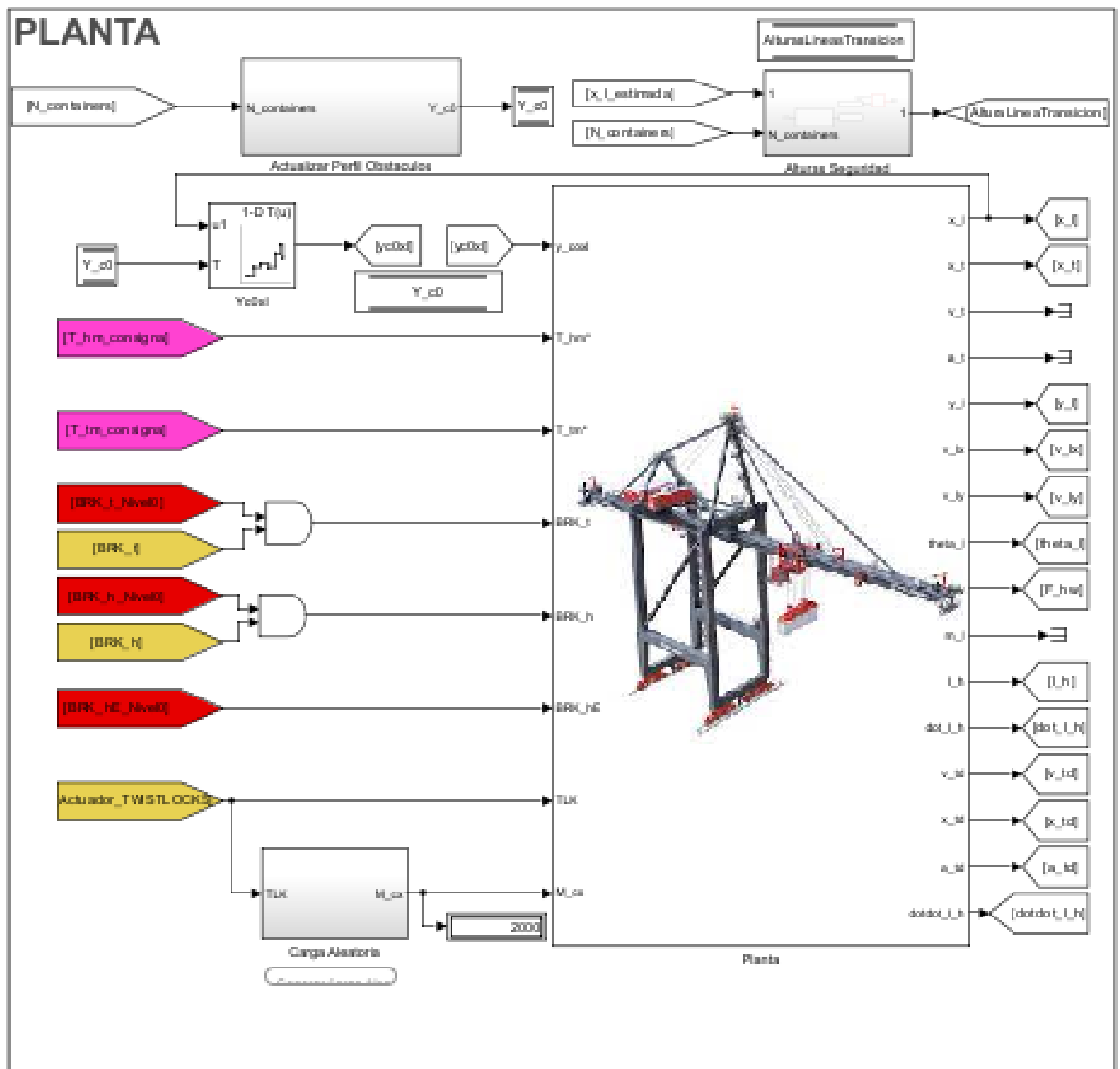
Se agrega una dinámica de primer orden a los distintos tipos de frenos a fin de evitar bucles algebraicos en la simulación.

Las ecuaciones correspondientes a la dinámica de las cajas reductoras de los accionamientos se reducen a solo una reflejada al eje lento para reducir la cantidad de ecuaciones.

Tanto la configuración de contenedores en la bahía de carga del barco como en los carriles del muelle correspondientes a los camiones de transporte son configurables.

El perfil de obstáculos se implementa mediante una tabla de búsqueda de una dimensión, en donde a cada intervalo relevante en el eje x se ha definido la altura del perfil de obstáculos.

Para una mayor comprensión de la implementación de la dinámica, se ha subdividido a la planta en tres subsistemas: carro, izaje y carga. Todas las imágenes utilizadas tienen por objeto representar a dichos subsistemas y sus componentes, no representan componentes reales de este tipo de grúas.



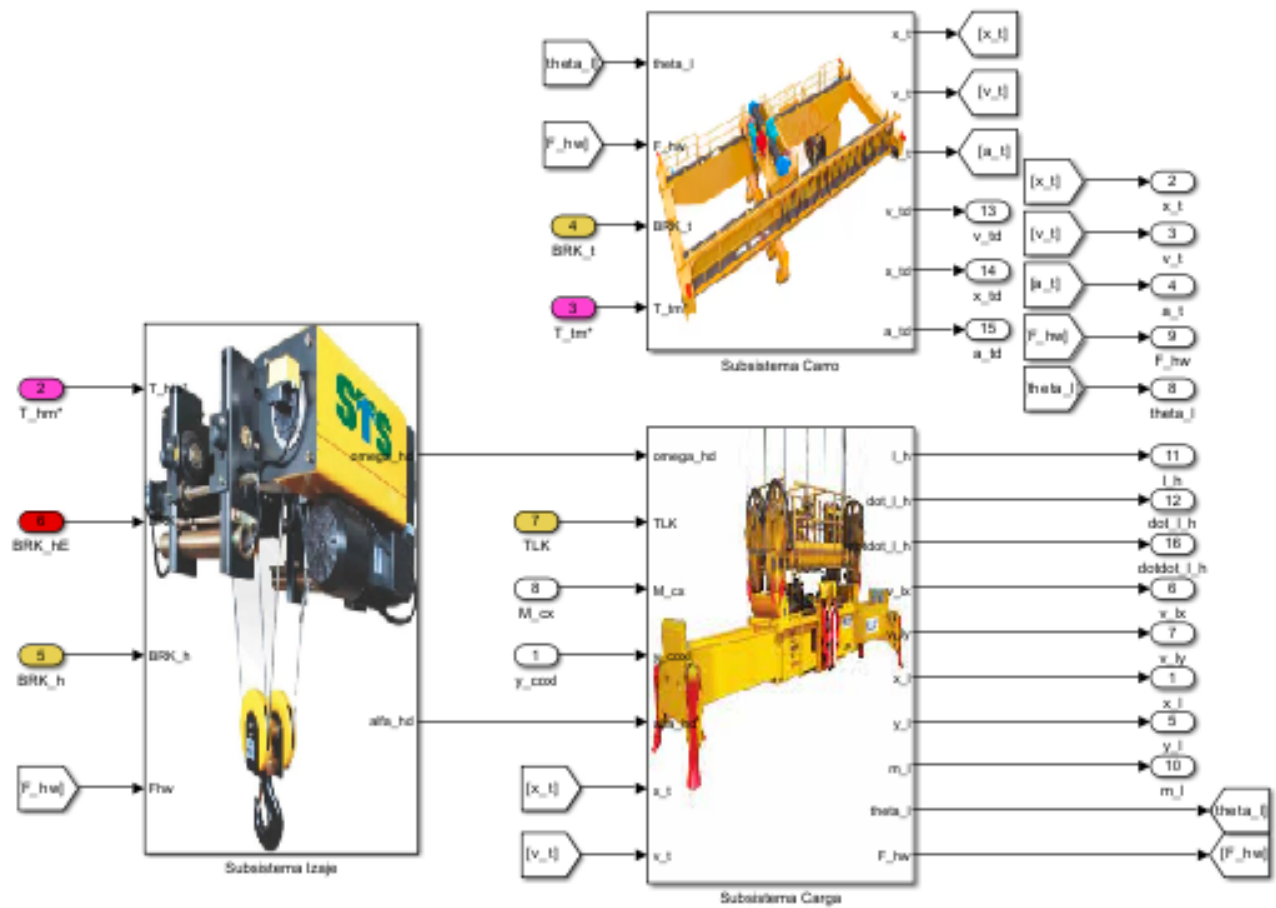


Figura 2: Subsistemas de la grúa en Simulink

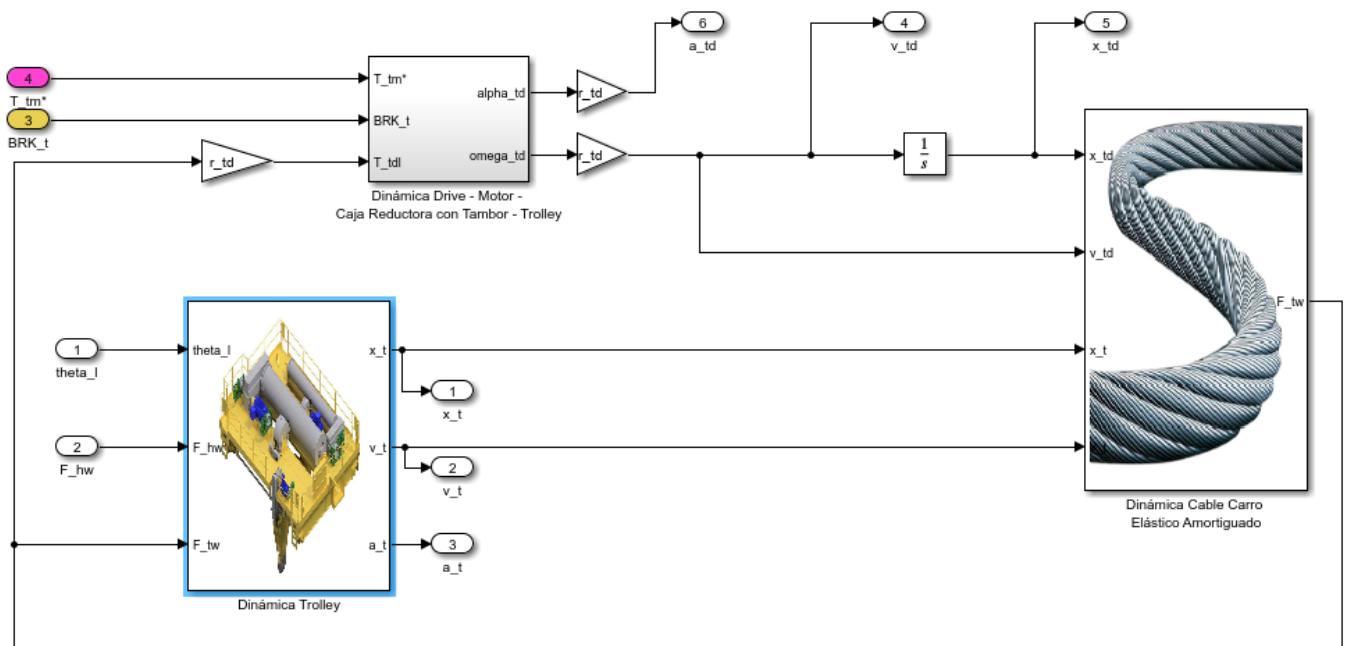


Figura 3: Subsistemas de carro en Simulink

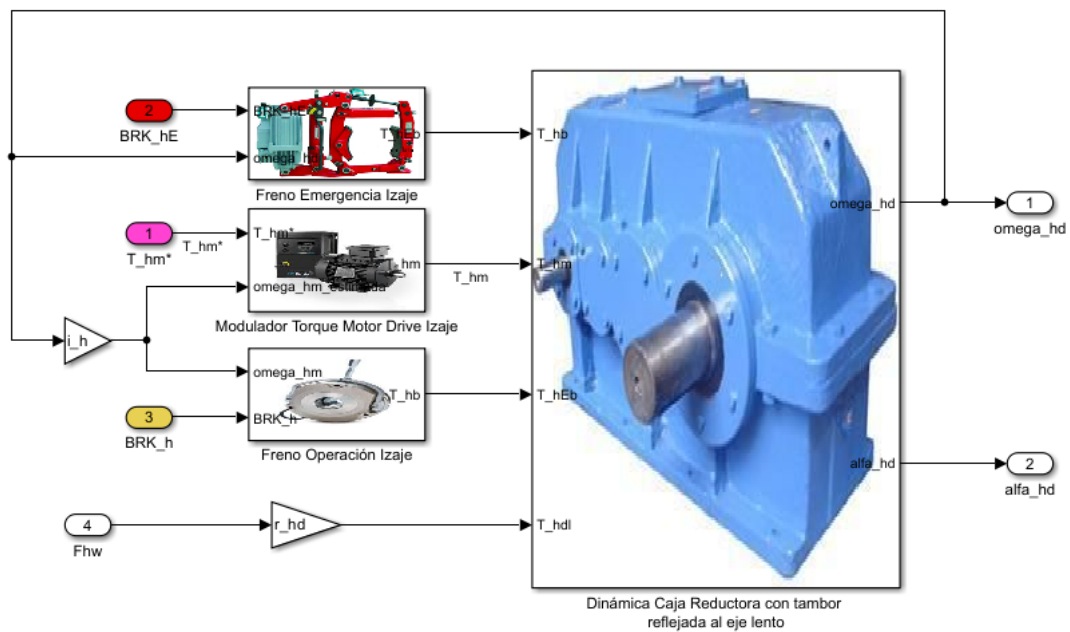


Figura 4: Subsistemas de izaje en Simulink

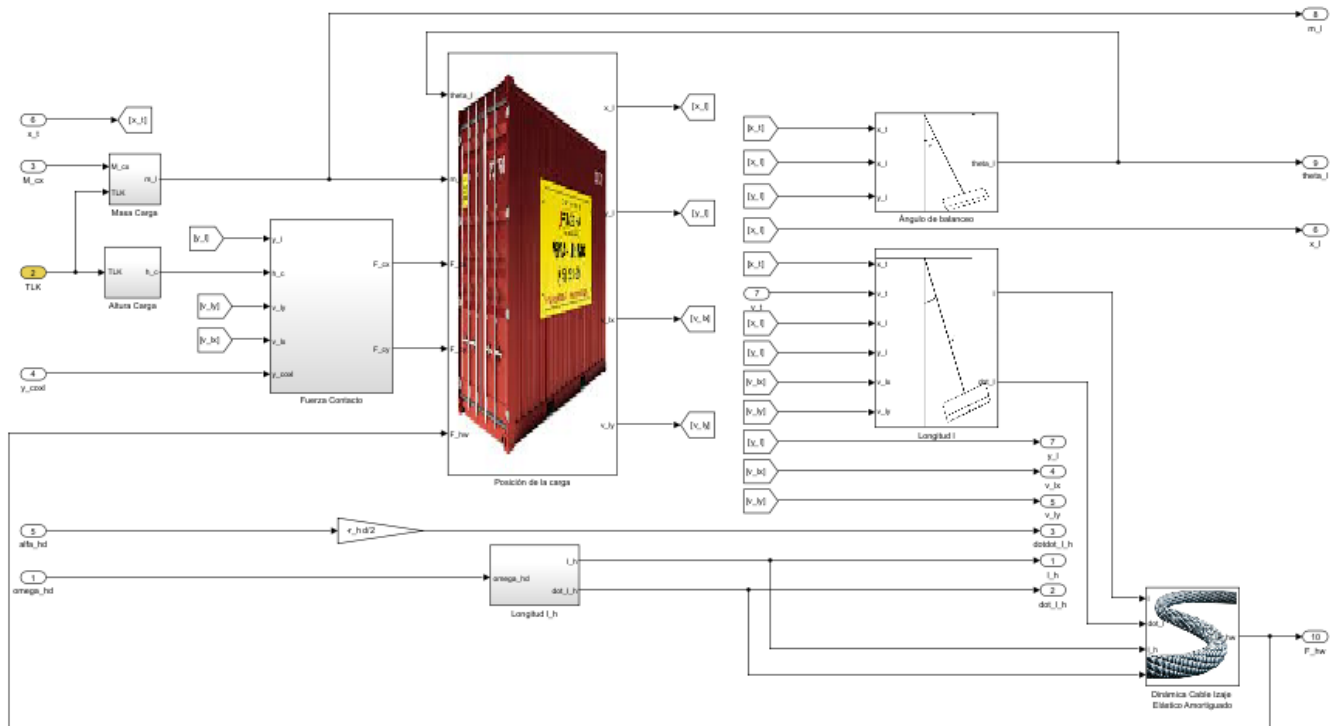


Figura 5: Subsistemas de carga en Simulink

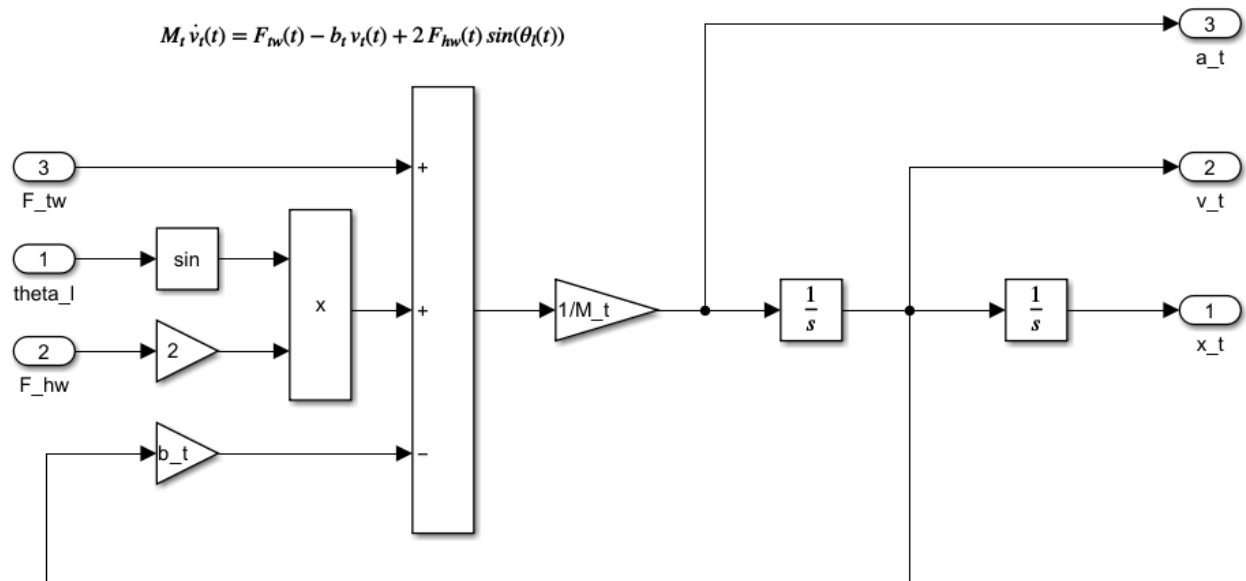


Figura 6: Dinámica de carro en Simulink

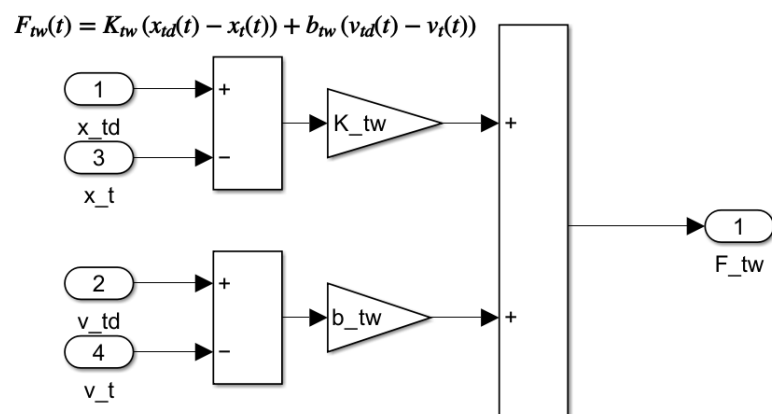


Figura 7: Dinámica de cable elástico de accionamiento de carro en Simulink

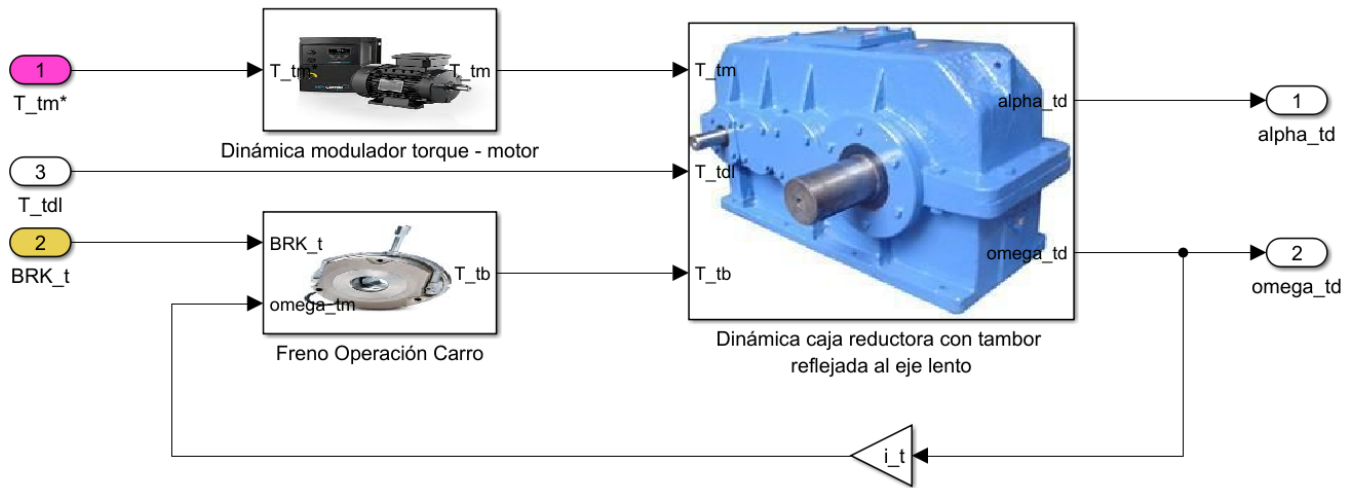


Figura 8: Dinámica de accionamiento de carro en Simulink

$$\tau_{tm} \dot{T}_{tm}(t) = T_{tmLim}^*(t) - T_{tm}(t)$$

$$T_{tmLim}^*(t)(k T_{s2} \leq t < (k+1) T_{s2}) = \min(T_{tmMax}, \max(-T_{tmMax}, T_{tm}^*(k T_{s2})))$$

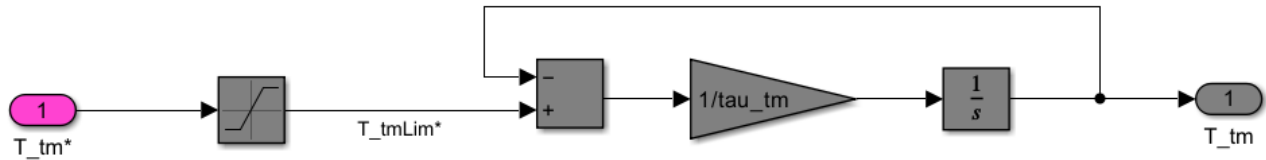


Figura 9: Dinámica modulador torque - motor drive accionamiento carro en Simulink

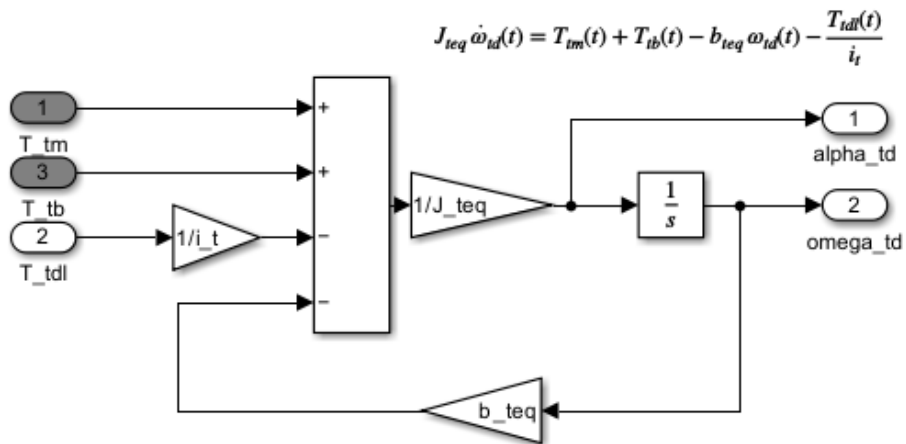


Figura 10: Dinámica caja reductora accionamiento carro en Simulink

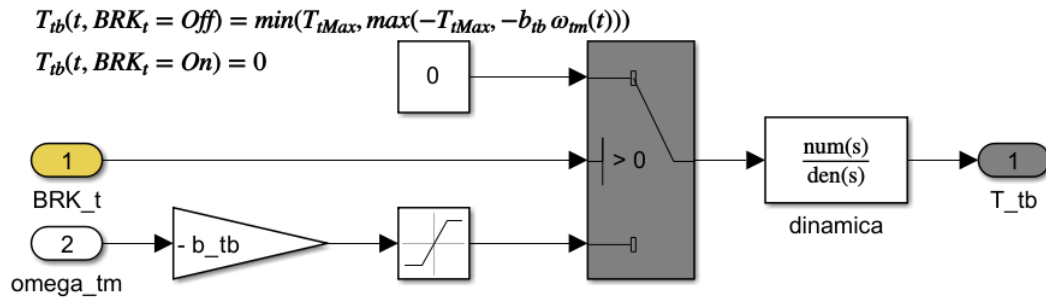


Figura 11: Dinámica freno de operación accionamiento carro en Simulink

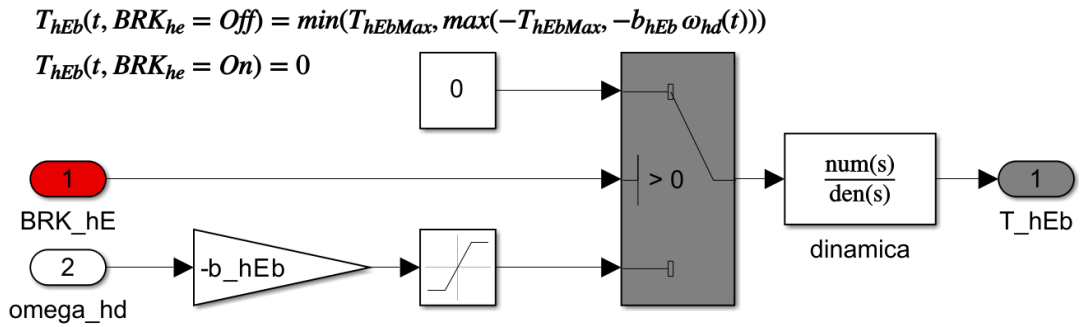


Figura 12: Dinámica freno de emergencia izaje en Simulink

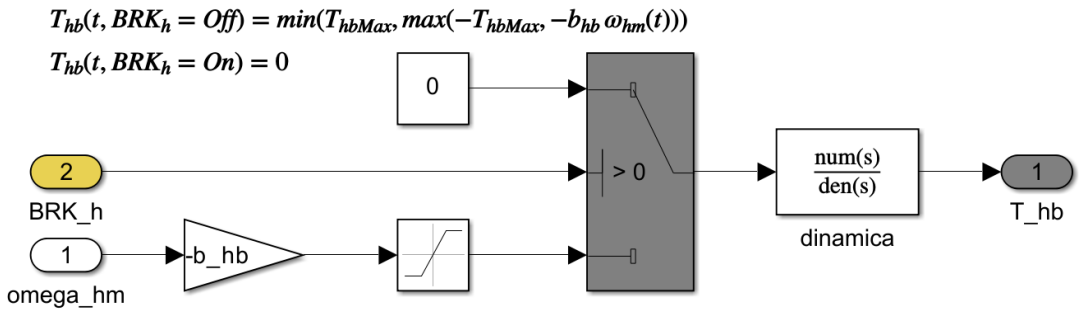


Figura 13: Dinámica freno de operacion izaje en Simulink

$$J_{heq} \dot{\omega}_{hd}(t) = T_{hm}(t) + T_{hb}(t) + \frac{T_{hEb}(t)}{i_h} - b_{heq} \omega_{hd}(t) - \frac{T_{hdl}(t)}{i_h}$$

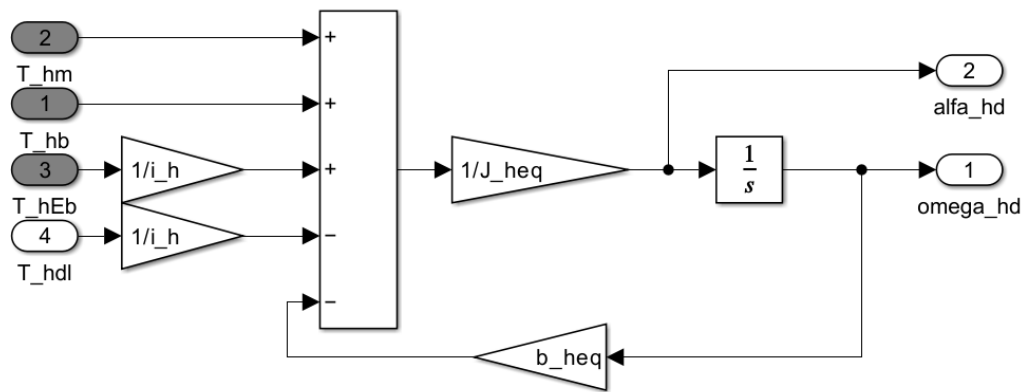


Figura 14: Dinámica caja reductora accionamiento izaje en Simulink

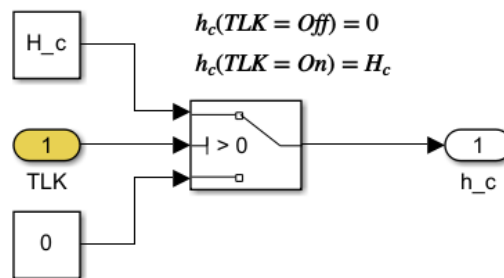


Figura 15: Longitud del cabezal en Simulink

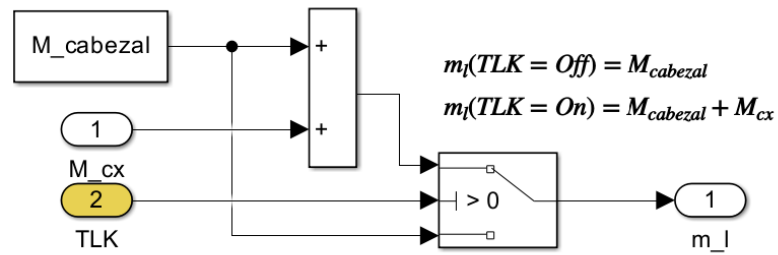


Figura 16: Longitud de la carga en Simulink

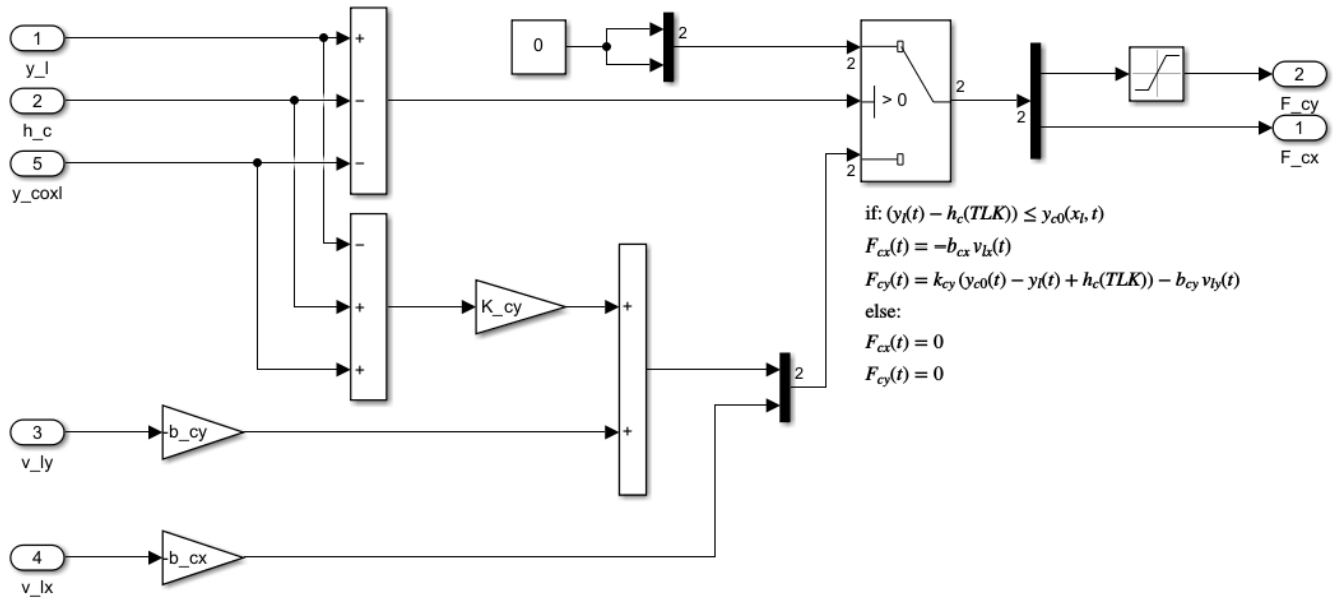


Figura 17: Fuerzas de contacto de la carga en Simulink

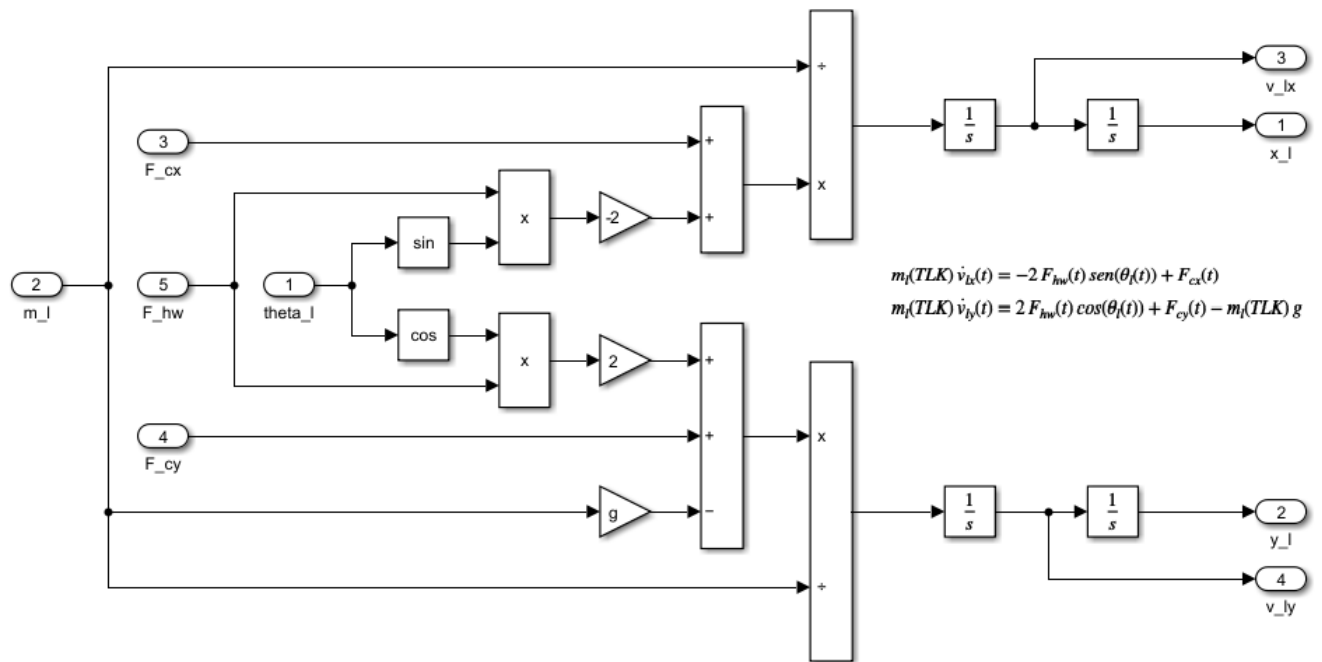


Figura 18: Movimiento de la carga en Simulink

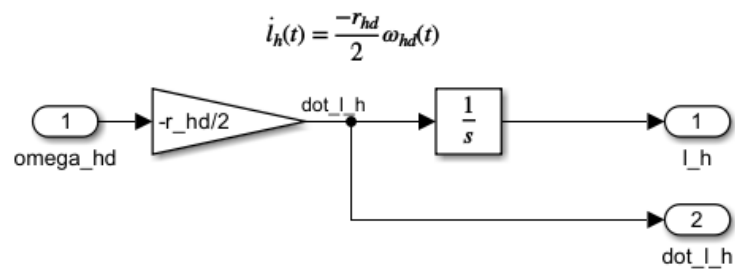


Figura 19: Longitud l_h en Simulink

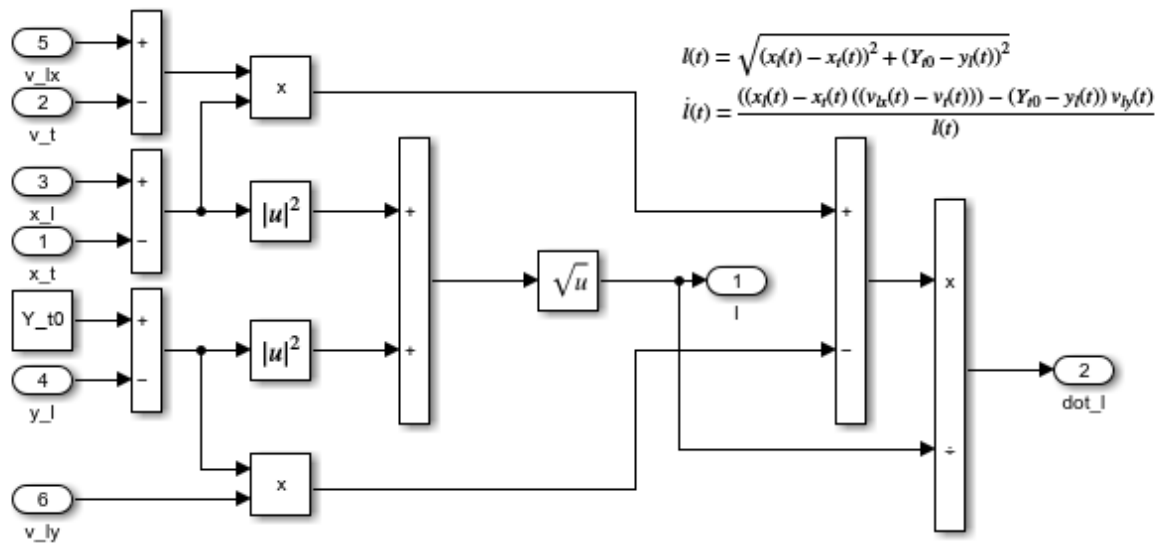


Figura 20: Cálculo de la longitud l en Simulink

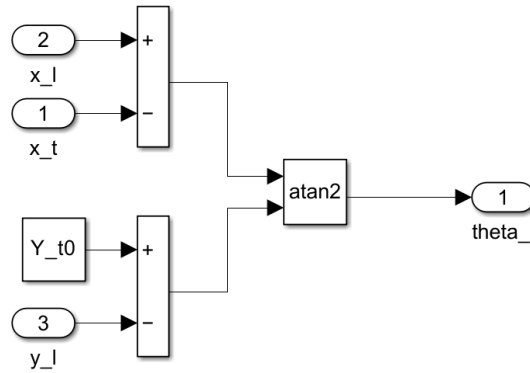


Figura 21: Cálculo del ángulo de balanceo de la carga en Simulink

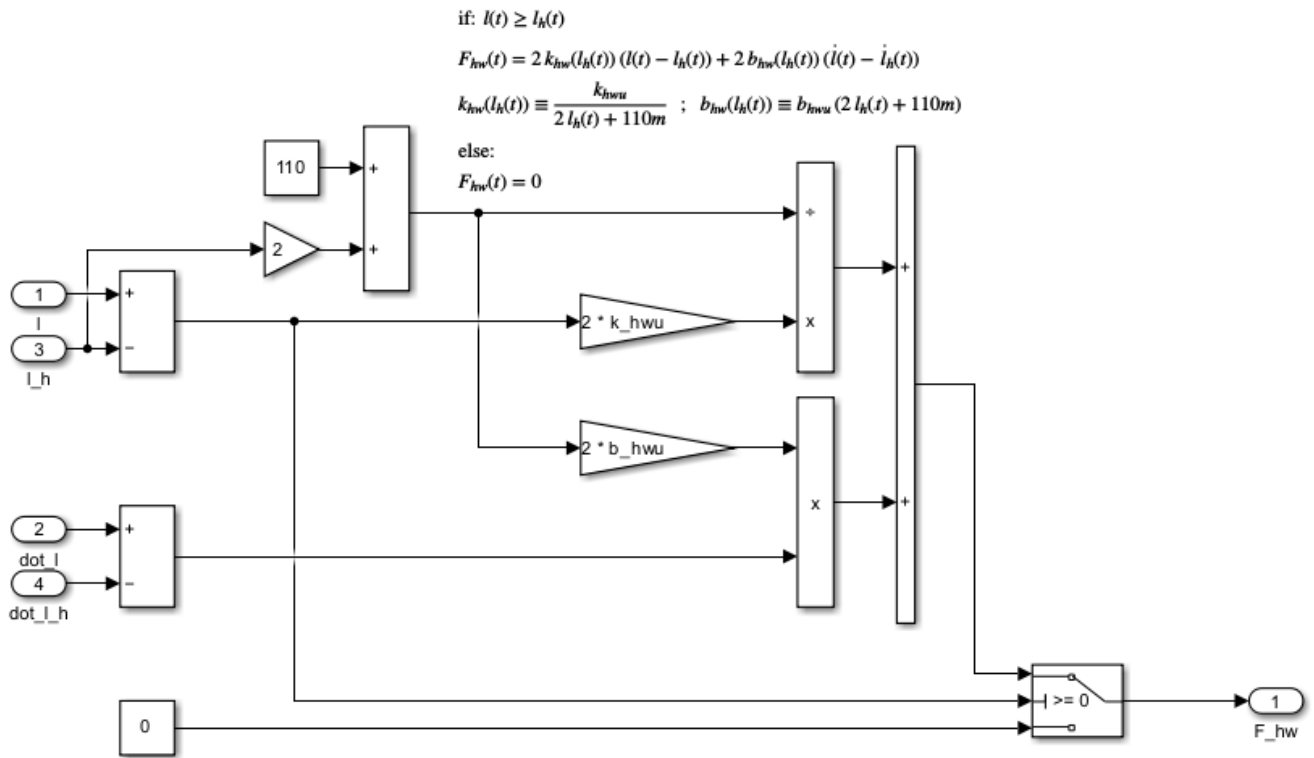


Figura 22: Dinámica de cable elástico de accionamiento de izaje en Simulink

2.1.2 NIVEL 2. CONTROL REGULADOR

REPLANTEO DE LAS ECUACIONES DINÁMICAS DEL SISTEMA:

A partir de reemplazar (7.e') de [1] en (7.d) de [1]

$$T_{tml} = \frac{1}{i_t} [J_{td} \dot{\omega}_{td}(t) + b_{td} \omega_{td}(t) + T_{tdl}(t)] \quad (1)$$

Reemplazando esta expresión en (7.f) de [1] sin considerar el torque de frenado

$$J_{teq} \dot{\omega}_{td}(t) = T_{tm}(t) - b_{teq} \omega_{td}(t) - \frac{T_{tdl}(t)}{i_t} \quad (2)$$

Siendo

$$J_{teq} = \frac{J_{td} + J_{tm+tb}i_t^2}{i_t} \quad (3)$$

$$J_{beq} = \frac{b_{td} + b_{tm}i_t^2}{i_t} \quad (4)$$

La ecuación (2) representa la dinámica conjunta de la caja de reducción del sistema de accionamiento de carro y su respectivo tambor.

De manera similar, la dinámica conjunta de la caja reductora del accionamiento de izaje resulta ser

$$J_{heq}\dot{\omega}_{hd}(t) = T_{hm}(t) - b_{heq}\omega_{hd}(t) - \frac{T_{hdl}(t)}{i_h} \quad (5)$$

Siendo

$$J_{heq} = \frac{J_{hd} + J_{hm+tb}i_h^2}{i_h} \quad (6)$$

$$J_{heq} = \frac{b_{hd} + b_{hm}i_h^2}{i_h} \quad (7)$$

Las ecuaciones (4.a) y (4.b) de [1] representan la dinámica de la carga, a partir de ellas se concluye

$$2F_{hw}(t) = -m_l\ddot{l}(t) - m_l\ddot{x}_t(t)\sin\theta_l(t) + m_ll(t)\dot{\theta}_l(t)^2 + m_lg\cos\theta_l(t) \quad (8)$$

$$m_l\ddot{x}_t(t)\cos\theta_l(t) + m_ll(t)\ddot{\theta}_l(t) + 2m_l\dot{l}(t)\dot{\theta}_l(t) + m_lg\sin\theta_l(t) = 0 \quad (9)$$

En función de (2) y (8), de (7.a) y (7.c') de [1] y considerando además que los cables de accionamiento de carro e izaje son lo suficientemente rígidos de modo que $v_{td} \approx v_t$ y $l_h \approx l$

$$(M_{teqTOT} + m_l)\ddot{x}_{td}(t) + m_ll_h(t)\cos\theta_l(t)\ddot{\theta}_l(t) + m_l\sin\theta_l(t)\ddot{l}_h(t) + b_{teqTOT}\dot{x}_{td}(t) + 2m_l\cos\theta_l(t)\dot{l}_h(t)\dot{\theta}_l(t) - m_l\sin\theta_l(t)l_h(t)\dot{\theta}_l(t)^2 = f_x \quad (10)$$

$$M_{teqTOT} = M_t + \frac{J_{teq}i_t}{r_{td}^2} \quad (11)$$

$$b_{teqTOT} = b_t + \frac{b_{teq}i_t}{r_{td}^2} \quad (12)$$

$$f_x \equiv \frac{T_{tm}(t)i_t}{r_{td}} \quad (13)$$

De manera similar, podemos expresar

$$(J_{teqTOT} + m_l)\ddot{l}_h(t) + m_l\sin\theta_l(t)\ddot{x}_{td}(t) + b_{heqTOT}\dot{l}_h(t) - m_ll_h(t)\dot{\theta}_l(t)^2 - m_lg\cos\theta_l(t) = f_y \quad (14)$$

$$J_{heqTOT} = \frac{4J_{heq}i_h}{r_{hd}^2} \quad (15)$$

$$b_{heqTOT} = \frac{4b_{heq}i_h}{r_{hd}^2} \quad (16)$$

$$f_y \equiv \frac{-2T_{hm}(t)i_h}{r_{hd}} \quad (17)$$

$$m_l\ddot{x}_{td}(t)\cos\theta_l(t) + m_ll_h(t)\ddot{\theta}_l(t) + 2m_l\dot{l}_h(t)\dot{\theta}_l(t) + m_lg\sin\theta_l(t) = 0 \quad (18)$$

Las ecuaciones (10), (14) y (18) constituyen las ecuaciones dinámicas en las cuales se basa el desarrollo del control regulador.

LEY DE CONTROL DEL CONTROL REGULADOR

Con la finalidad de cumplir con las especificaciones de desempeño requeridas en [1], se procede al desarrollo de las leyes de control según [2]. El principio de funcionamiento se basa en introducir suficiente amortiguamiento a la dinámica del ángulo de balanceo expresada por (18).

La trayectoria deseada $r \equiv (r_x \ r_l \ r_\theta)^T$ de las variables $q \equiv (x_{td} \ l_h \ \theta_l)^T$ se modifica a r_m

$$r_m \equiv \begin{pmatrix} r_x + k_{as}\Theta_I \\ r_l \\ r_\theta \end{pmatrix} \quad (19)$$

Donde $\Theta_I \equiv \int_0^t (\theta_l / \cos \theta_l) d\tau$, con $\|\theta_l\| < \pi/2$ y $k_{as} > 0$ denominada como ganancia de control de oscilación. Para minimizar entonces el ángulo de oscilación, se establece $r_\theta = 0$.

El error de seguimiento de consigna viene dado por

$$e \equiv \begin{pmatrix} e_x \\ e_l \\ e_\theta \end{pmatrix} = \begin{pmatrix} r_x + k_{as}\Theta_I - x_{td} \\ r_l - l_h \\ -\theta_l \end{pmatrix} \quad (20)$$

Podemos expresar entonces la ley de control como:

$$f_c \equiv (f_x \ f_y)^T = W_c p + K_d \dot{e}_c + K_p e_c \quad (21)$$

Siendo

$$W_c p \equiv M_{cc} \ddot{m}_c + C_{cc} \dot{m}_c + D_{cc} \dot{q}_c + g_c \quad (22)$$

Donde

$$M_{cc} = \begin{bmatrix} M_{teqTOT} + m_l & m_l \sin \theta_l \\ m_l \sin \theta_l & J_{heqTOT} + m_l \end{bmatrix} \quad (23)$$

$$C_{cc} = \begin{bmatrix} 0 & m_l \cos \theta_l \dot{\theta}_l \\ 0 & 0 \end{bmatrix} \quad (24)$$

$$D_{cc} = \begin{bmatrix} b_{teqTOT} & 0 \\ 0 & b_{heqTOT} \end{bmatrix} \quad (25)$$

$$q_c = (x_{td} \ l_h)^T \quad (26)$$

$$r_{mc} = (r_x + k_{as}\Theta_I \ r_l)^T \quad (27)$$

$$g_c = (0 \ -m_l g)^T \quad (28)$$

$$p = (m_l \ M_{teqTOT} \ J_{heqTOT} \ b_{teqTOT} \ b_{heqTOT})^T \quad (29)$$

Donde W_c es la matriz regresora que se puede definir según la ley de control y p . El subíndice c hace referencia a las variables de control.

Considerando la dinámica descrita por (10), (14) y (18) y suponiendo que r_m , $\dot{r}_m$ y $\ddot{r}_m$ están suficientemente acotadas, $2\|\dot{r}_l\| \leq k_{as}$ para todo $t \geq 0$ y que siempre $r_l > 0$, entonces la ley de control planteada garantiza que $e, \dot{e} \in L_\infty$ si $\|\theta_l(0)\| < \pi/2$, $\|\dot{e}(0)\| < \delta_s$ y $\|e(0)\| < \delta_s$, para una constante positiva δ_s no infinita. Luego $e(t) \rightarrow 0$ y $\dot{e}(t) \rightarrow 0$ asintóticamente mientras $t \rightarrow \infty$ si $\ddot{r}_m = 0$, $\dot{r}_l = 0$, $\|\theta_l(0)\| < \pi/2$, $\|\dot{e}(0)\| < \delta_s$ y $\|e(0)\| < \delta_s$. El control propuesto entonces garantiza estabilidad en las zonas de aceleración y desaceleración y estabilidad asintótica en las zonas de velocidad constante y en la posición deseada.

En estos dos últimos casos, $\ddot{e}_x = 0 \rightarrow \ddot{r}_{mx} = \ddot{x}_{td}$.

$$\ddot{x}_{td} = \ddot{r}_x + k_{as} \left(\frac{1}{\cos \theta_l} + \frac{\theta_l \sin \theta_l}{\cos^2 \theta_l} \right) \dot{\theta}_l \quad (30)$$

Reemplazando esta última expresión en (18), la dinámica del ángulo de oscilación se vuelve

$$l_h \ddot{\theta}_l + k_{as} \left(1 + \frac{\theta_l \sin \theta_l}{\cos \theta_l} \right) \dot{\theta}_l + 2\dot{l}_h \dot{\theta}_l + g \sin \theta_l = -\cos \theta_l \ddot{x} \quad (31)$$

Para ángulos pequeños y l_h constante, podemos plantear

$$l_h \ddot{\theta}_l + k_{as} \dot{\theta}_l + g \theta_l \approx -\ddot{x} \quad (32)$$

En este caso, $\omega_n = \sqrt{g/l_h}$ entonces

$$\frac{k_{as}}{l_h} = 2\zeta \left(\sqrt{\frac{g}{l_h}} \right) \rightarrow k_{as} = 2\zeta \sqrt{g l_h} \quad (33)$$

Satisfaciendo así los requerimientos de las sección 4.2 de [1].

La demostración de la efectividad de la ley de control planteada se basa en el segundo teorema de estabilidad de Lyapunov, planteando

$$E(t) = \frac{1}{2} \dot{e}^T M \dot{e} + \frac{1}{2} e_c^T K_p e_c + m_l g l_h (1 - \cos \theta_l) \quad (34)$$

Donde

$$M = \begin{bmatrix} M_{teqTOT} + m_l & m_l \sin \theta_l & m_l l_h \cos \theta_l \\ m_l \sin \theta_l & J_{heqTOT} + m_l & 0 \\ m_l l_h \cos \theta_l & 0 & m_l l_h^2 \end{bmatrix} \quad (35)$$

Es la matriz de inercia del sistema conformado por (10), (14) y (18).

Para una derivación completa [2]. f

2.1.3 Implementación del control regulador en Simulink

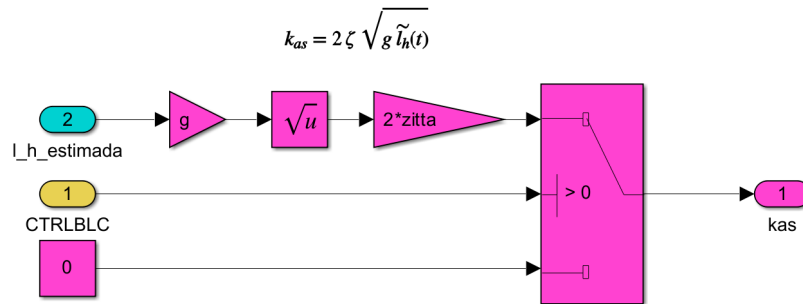


Figura 23: Cálculo de la ganancia k_{as} en Simulink

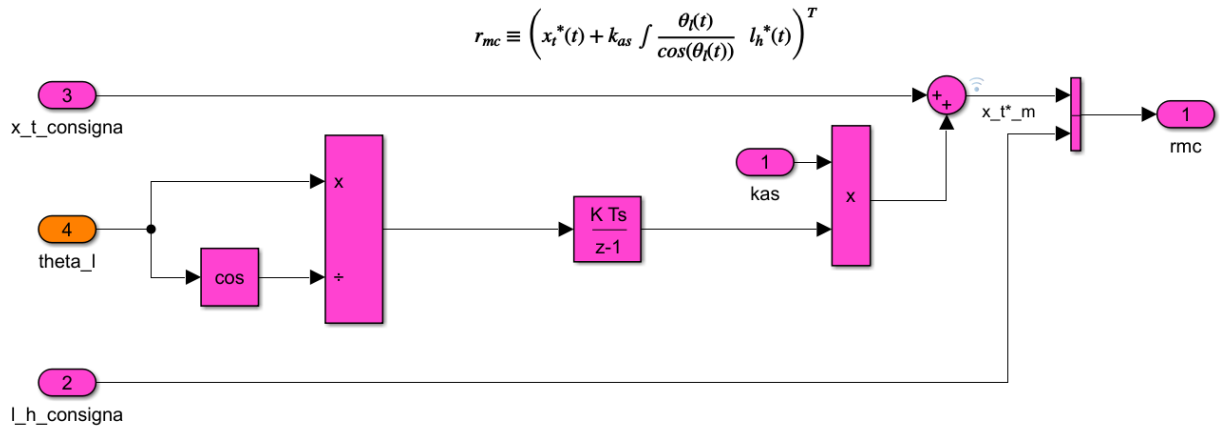


Figura 24: Cálculo r_{mc} en Simulink

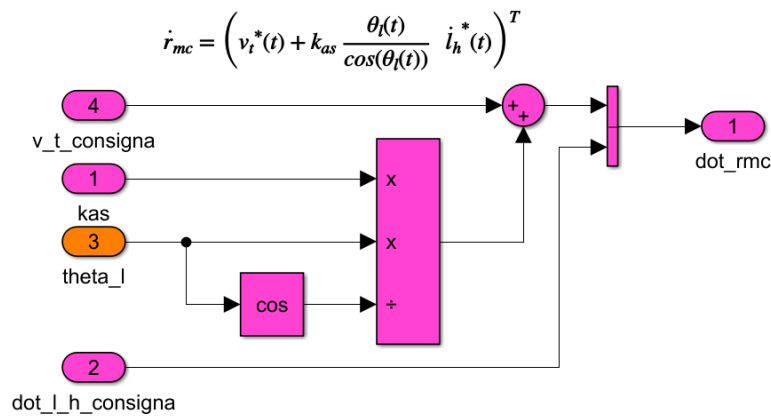


Figura 25: Cálculo $\dot{r}_{mc}$ en Simulink

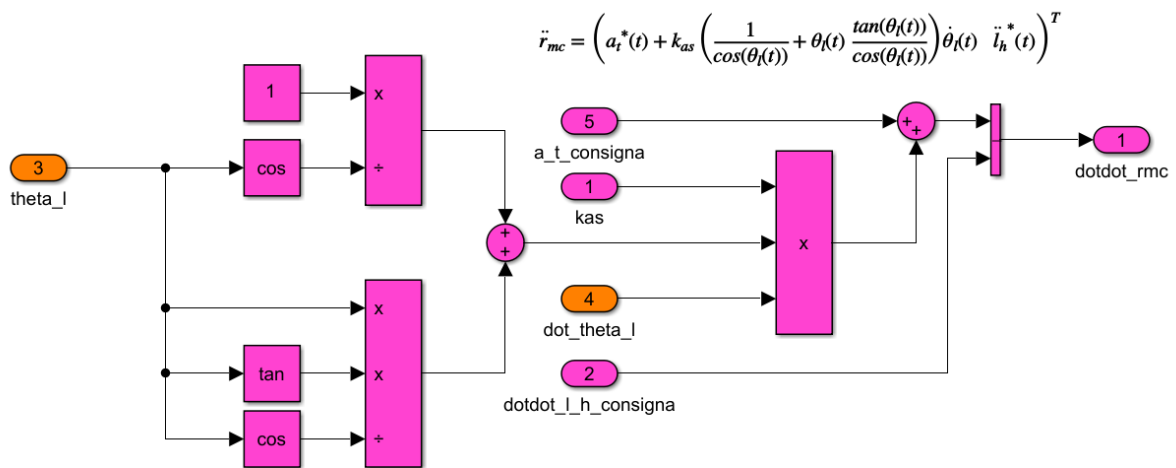


Figura 26: Cálculo $\ddot{r}_{mc}$ en Simulink

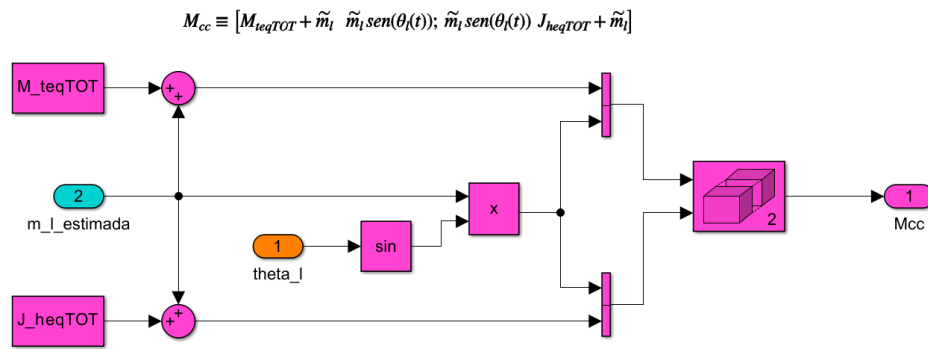


Figura 27: Cálculo M_{cc} en Simulink

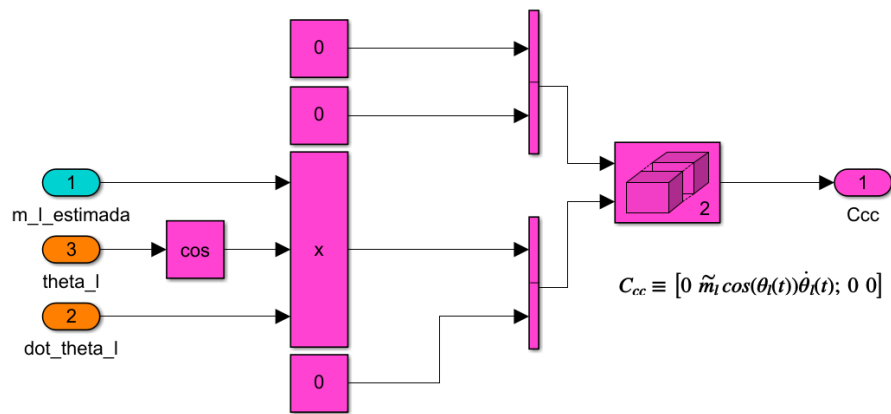


Figura 28: Cálculo C_{cc} en Simulink

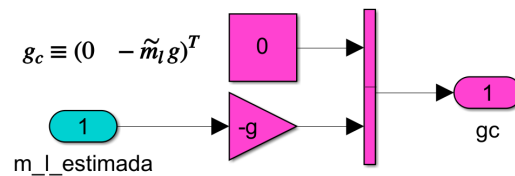


Figura 29: Cálculo g_c en Simulink

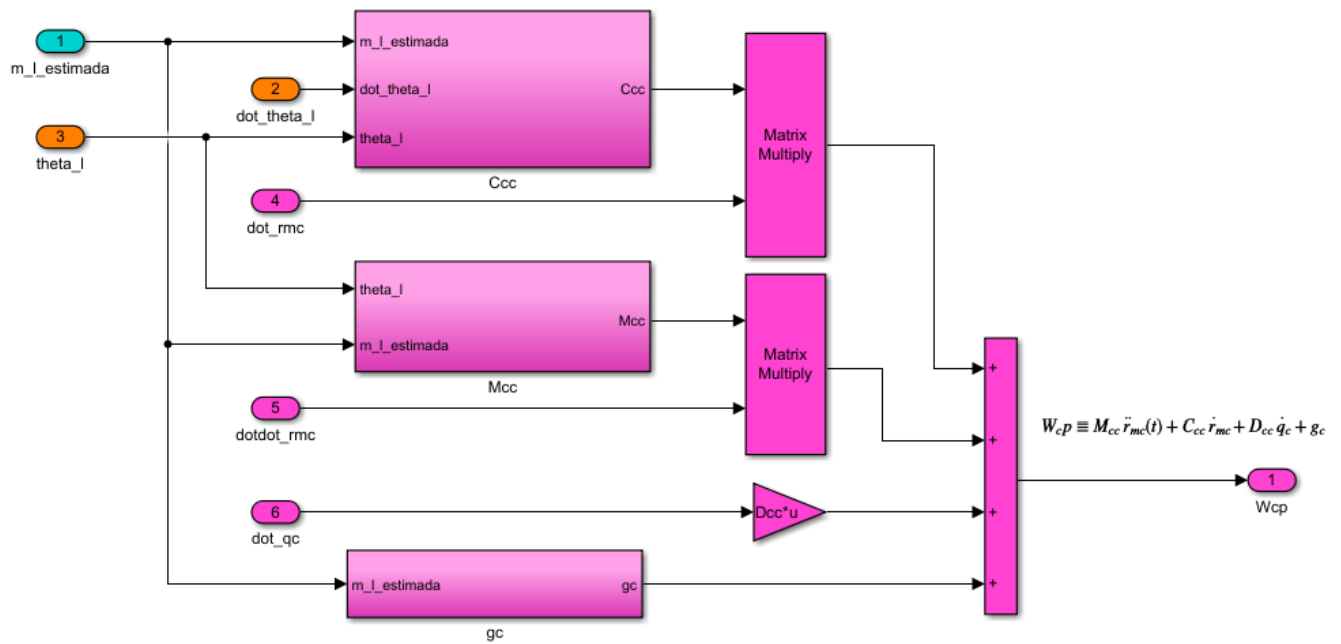


Figura 30: Cálculo W_{cp} en Simulink

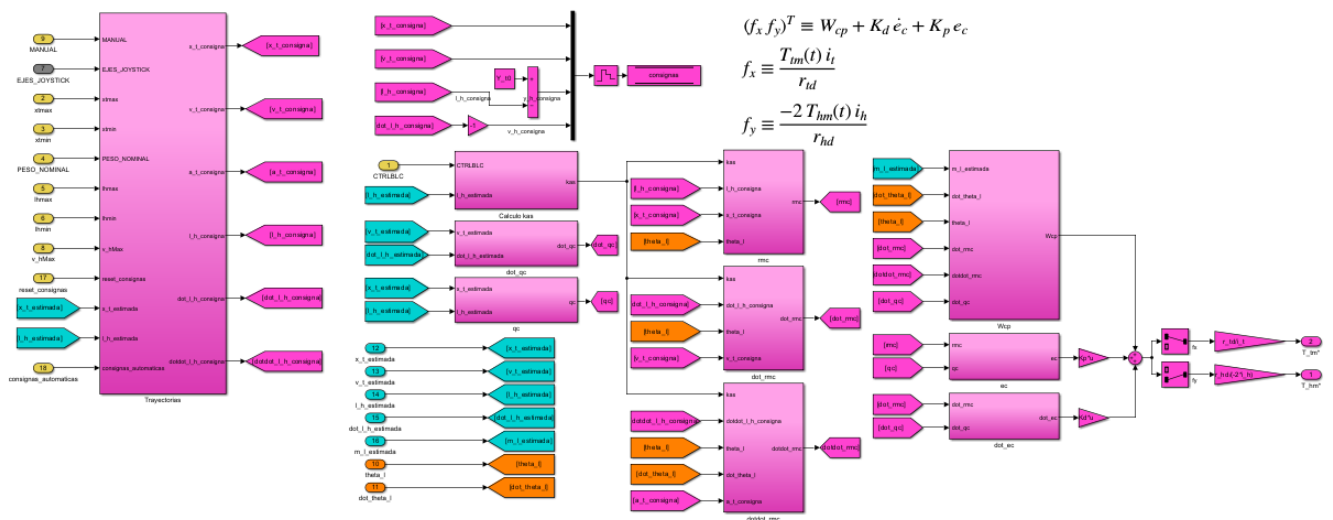


Figura 31: Control regulador completo en Simulink

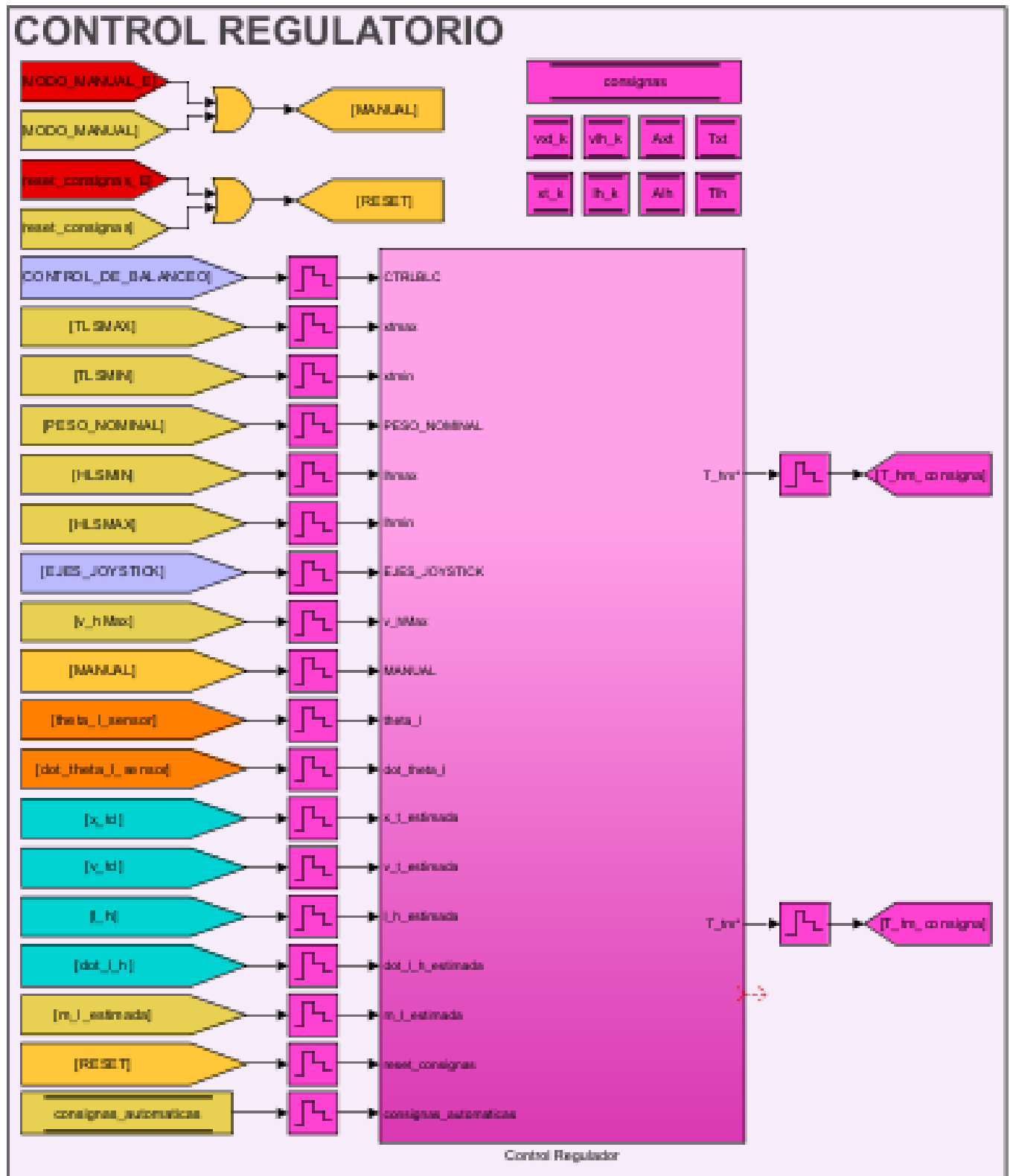


Figura 32: Control regulador como bloque en Simulink

En las imágenes siguientes puede apreciarse la efectividad de la ley de control implementada en simulink.

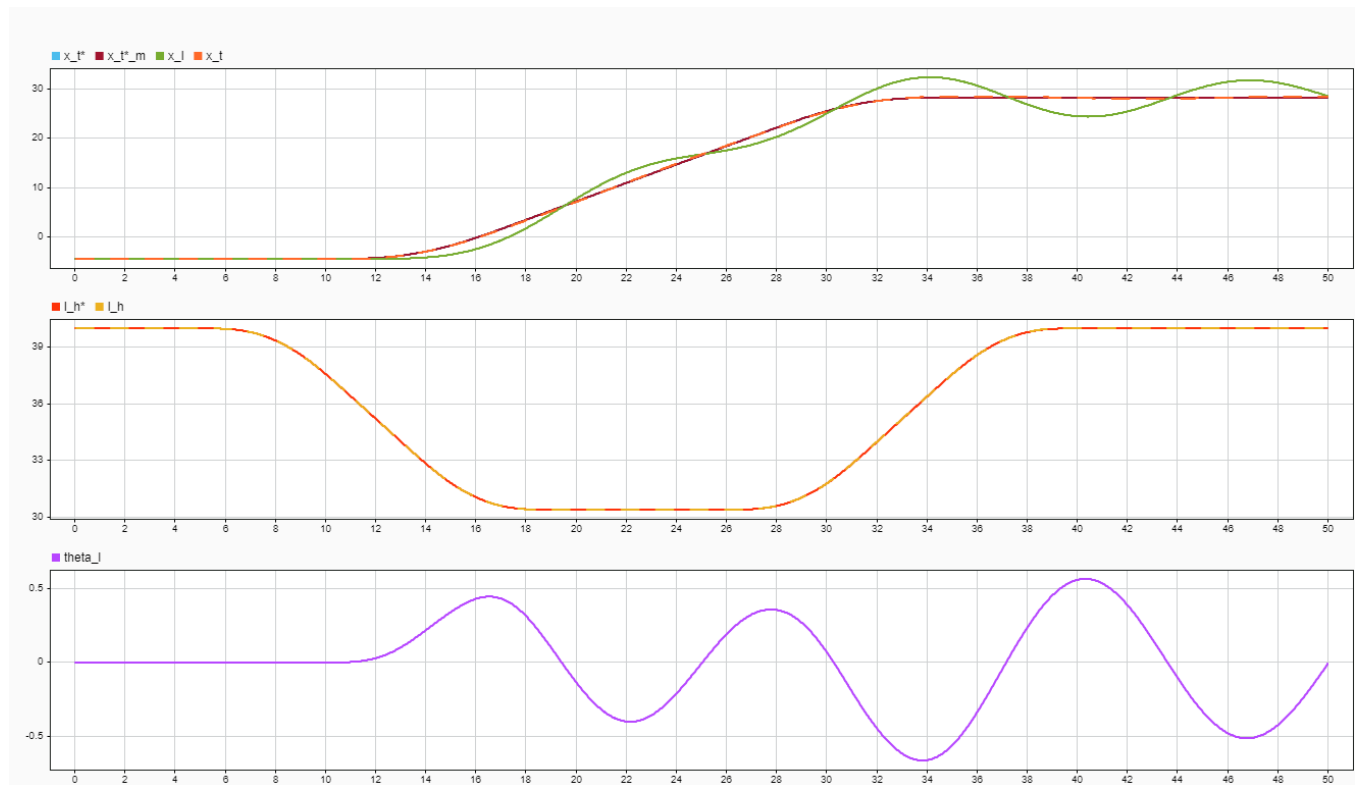


Figura 33: Control de balanceo apagado

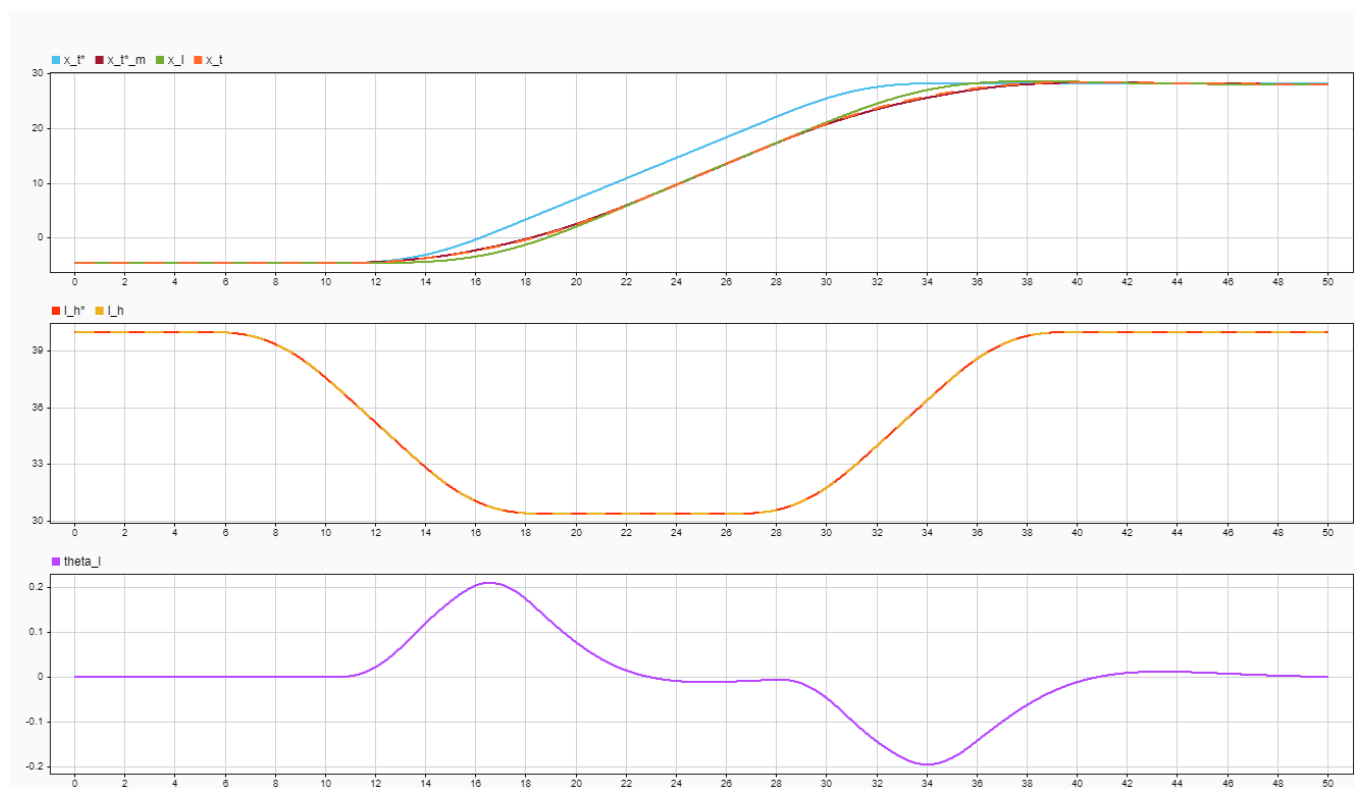


Figura 34: Control de balanceo encendido

Debido a (19), se introduce en la consigna un pequeño error de estado estacionario que, dado el contexto de la aplicación incluido su funcionamiento semiautomático, carece de relevancia.

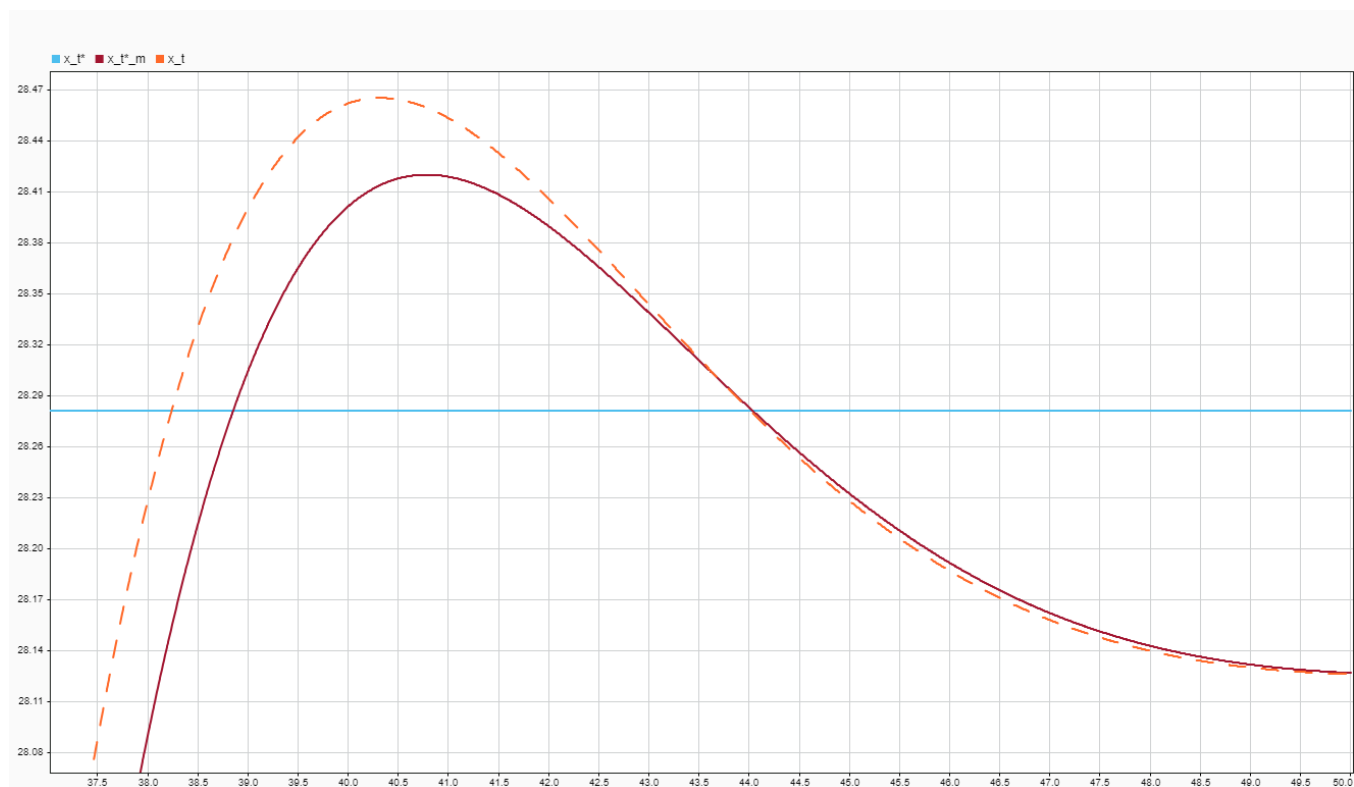


Figura 35: *Error de estado estacionario con control de balanceo*

2.1.4 Algoritmo de consignas manuales

Con la finalidad de asegurar un movimiento manual suave y una transferencia de mando “*bumpless*” en el paso de modo automático a manual, se utilizan perfiles de aceleración trapezoidal/triangular generados a partir de la consigna de velocidad deseada a través del *joystick* del operario, tomando en cuenta la consigna de velocidad y aceleración actual, satisfaciendo así la necesidades de la sección 4.3 de [1].

Siempre que se detecte un cambio en el estado actual de los mandos de movimiento del operario, actúa el siguiente algoritmo en conjunto con otras tareas especificadas por el control supervisor. El subíndice “a” significa “actual”, “d” significa “deseada”.

```

Algoritmo Generar_Perfil_Velocidad_S
Entrada: ta, vd, va, aa, aMax
Salida: perfil_a, tiempo_perfil

Definir tiempo_perfil como un vector de N elementos nulos
Definir perfil_a como un vector de N elementos nulos

delta_v <- vd - va
b <- signo(delta_v)
tjMax <- (b*aMax - aa) / (b*jMax)
tjMin <- aMax / jMax

delta_v_jerk <- aa * tjMax + 0.5 * b * jMax * tjMax^2
delta_v_jerk_total <- delta_v_jerk +
    b * aMax * tjMin - 0.5 * b * jMax * tjMin^2

Si |delta_v| > |delta_v_jerk_total| Entonces
    delta_v_constante <- |delta_v| - |delta_v_jerk_total|
    tc <- delta_v_constante / aMax
    aMaxAlcanzada <- b * aMax
Sino
    A <- b * jMax
    B <- 2 * aa
    C <- 0.5 * aa^2 / (b * jMax) - delta_v
    tt1 <- (-B + sqrt(B^2 - 4*A*C)) / (2*A)
    tt2 <- (-B - sqrt(B^2 - 4*A*C)) / (2*A)

    Si tt1 es real y tt1 > 0 Entonces
        tjMax <- tt1
    Sino Si tt2 es real y tt2 > 0 Entonces
        tjMax <- tt2
    Fin Si

    tc <- 0
    tjMin <- tt + aa / (b * jMax)
    aMaxAlcanzada <- aa + b * jMax * tjMax
Fin Si

T <- tjMax + tc + tjMin

Definir tiempo_perfil como un vector de N valores
entre 0 y T
Definir perfil_a como un vector de N ceros

Para i desde 1 hasta N Hacer
    Si tiempo_perfil[i] <= tjMax Entonces
        perfil_a[i] <- aa
        + b * jMax * tiempo_perfil[i]
    Sino Si tiempo_perfil[i] <= tjMax + tc Entonces
        perfil_a[i] <- aMaxAlcanzada
    Sino
        perfil_a[i] <- aMaxAlcanzada
        - b * jMax * (tiempo_perfil[i] - tjMax - tc)
    Fin Si
Fin Para

tiempo_perfil <- tiempo_perfil + ta

Fin Algoritmo

```

En la figura 36 se observan las consignas enviadas al control regulador por el algoritmo en función de la velocidad requerida por el mando del operario. En la figura 37 se observa la implementación en simulink.

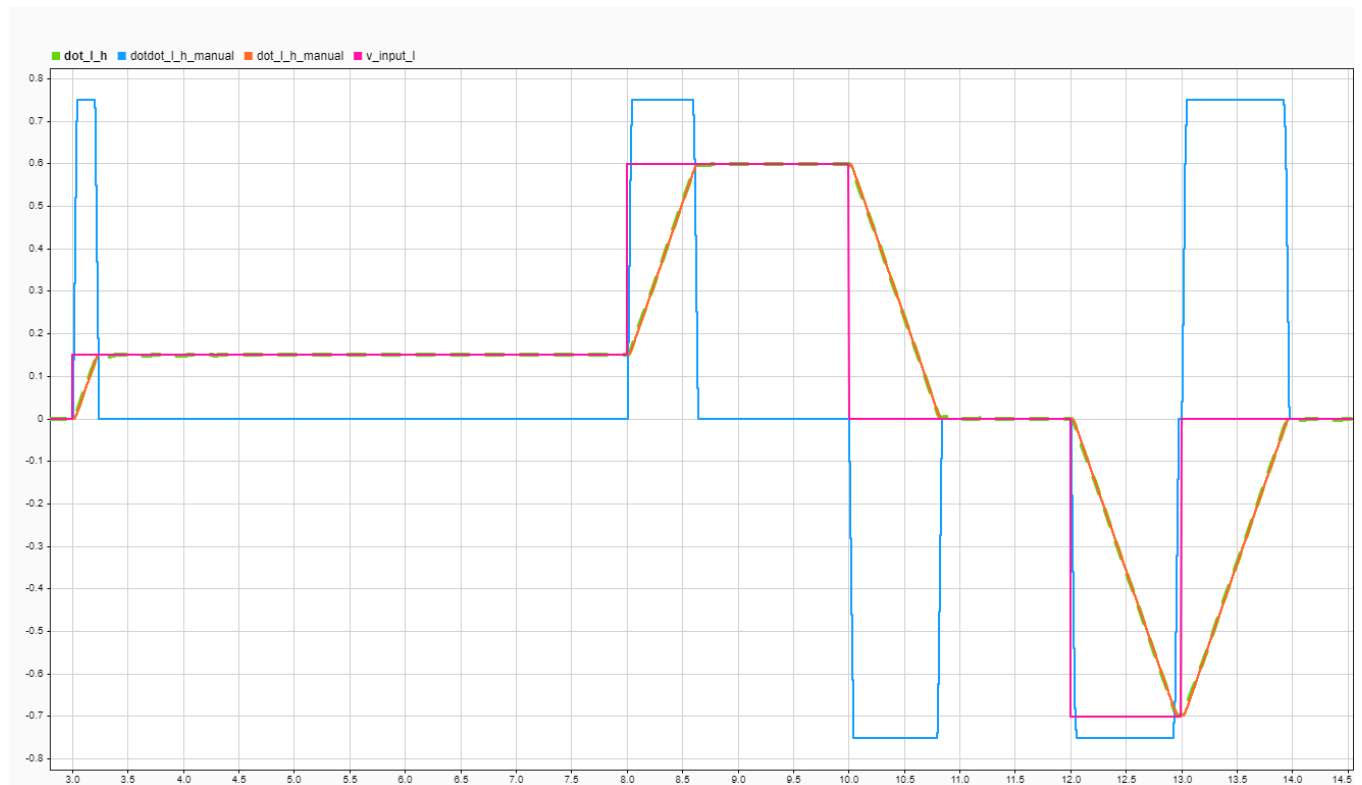


Figura 36: Trayectorias manuales

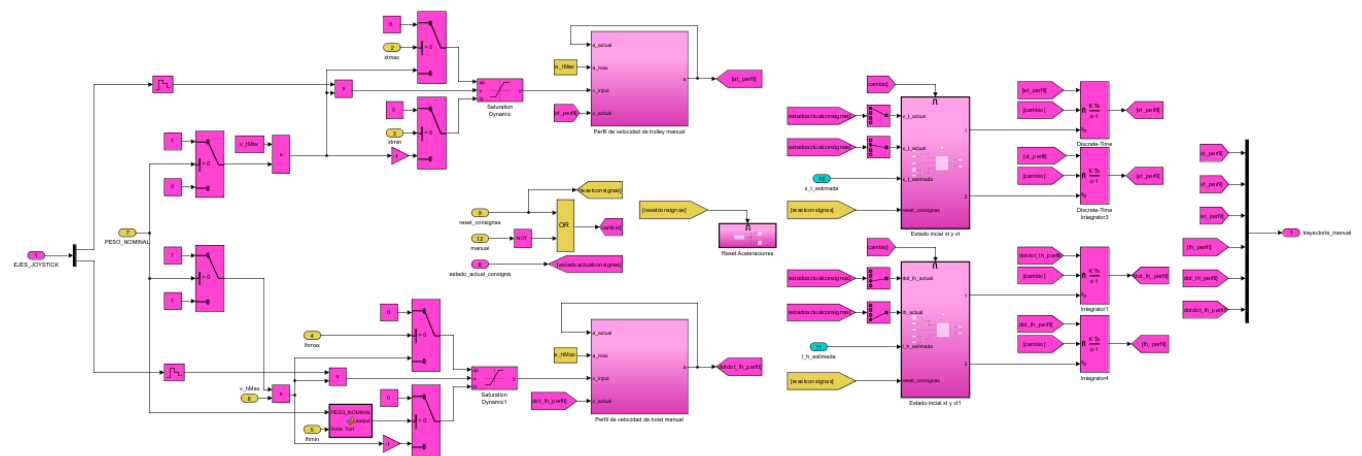


Figura 37: Cálculo de trayectorias manuales en simulink

2.1.5 Implementación del control de seguridad en Simulink

El control de seguridad tiene por objeto llevar al sistema a un estado seguro ante la detección de una falla de cualquier tipo. Su implementación debe ser sencilla y confiable. Se contemplan tres tipos de emergencias, aquella que se dan debido a la acción de un “watchdog”, aquella que se da debido al pulsador de emergencia del operario y aquellas que se dan debido a que la carga se encuentra en sus límites operativos.

Ante un estado de emergencia, el control de seguridad toma el mando del sistema, bloqueando por completo cualquier tipo de maniobra. La salida de dicho estado solo puede llevarse a cabo mediante un reinicio del sistema, simulando que la falla ha quedado atrás, devolviéndole el control manual al operador.

El automáta se implementa mediante la herramienta *Stateflow* de Simulink, como se muestra a continuación.

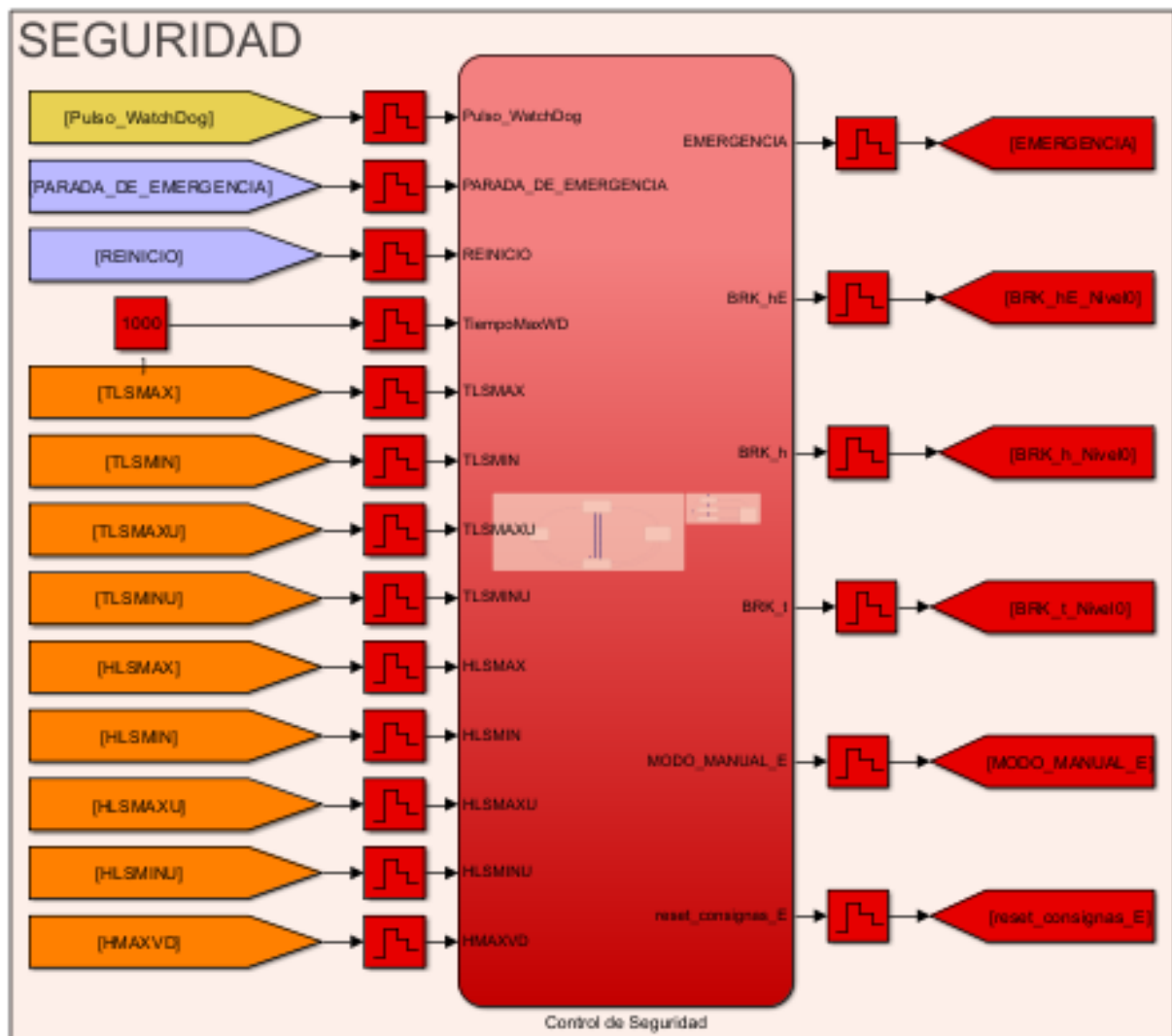


Figura 38: Control de seguridad como bloque en Simulink

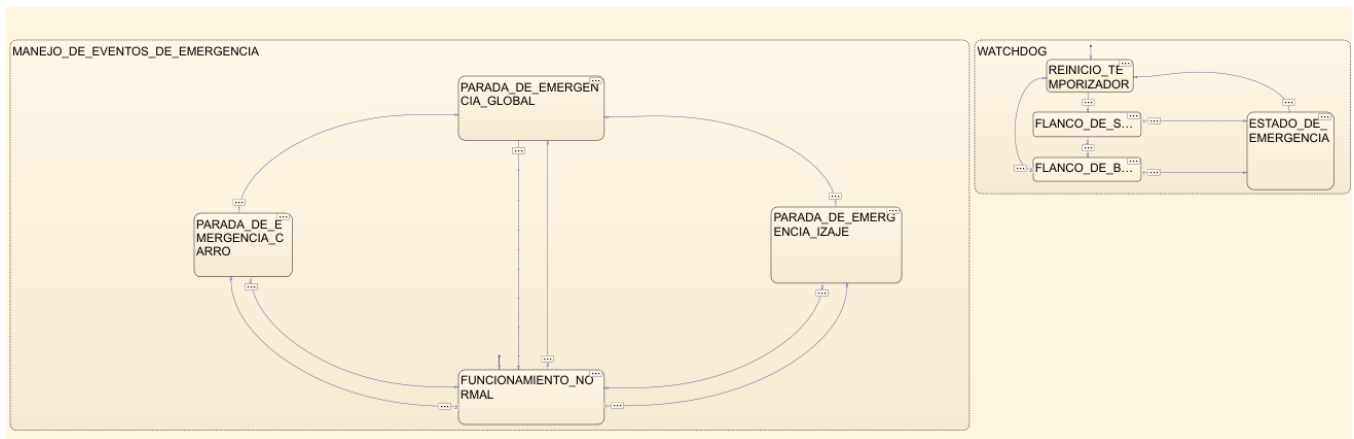


Figura 39: Implementación control de seguridad en Simulink

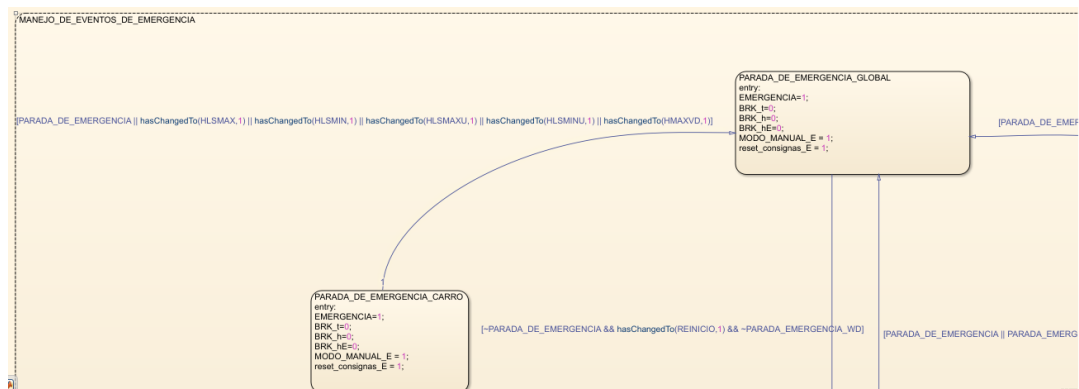


Figura 40: Manejo del estado de emergencia en Simulink (1)

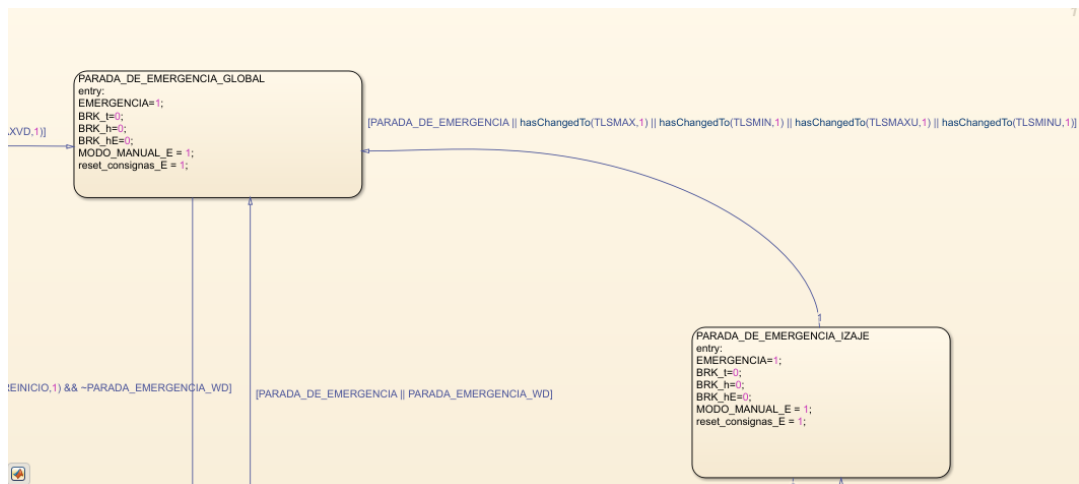


Figura 41: Manejo del estado de emergencia en Simulink (2)

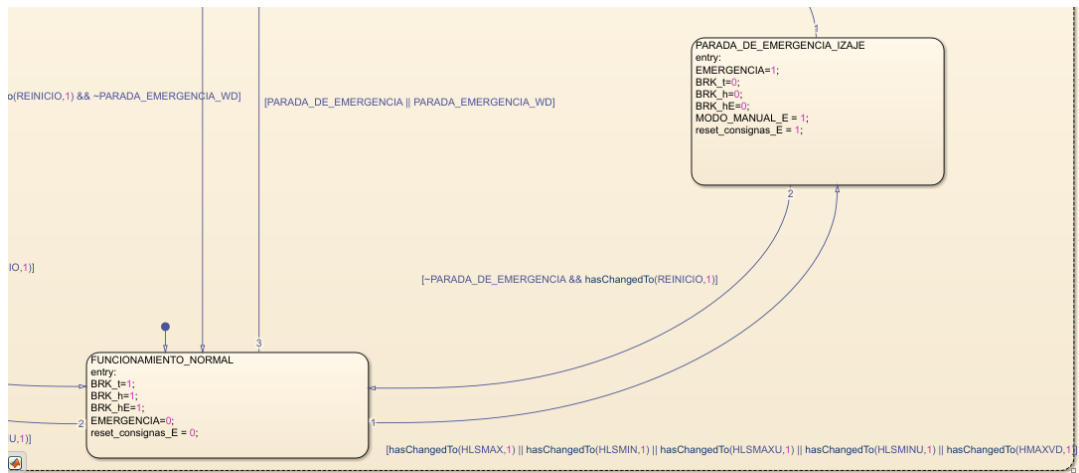


Figura 42: Manejo del estado de emergencia en Simulink (3)

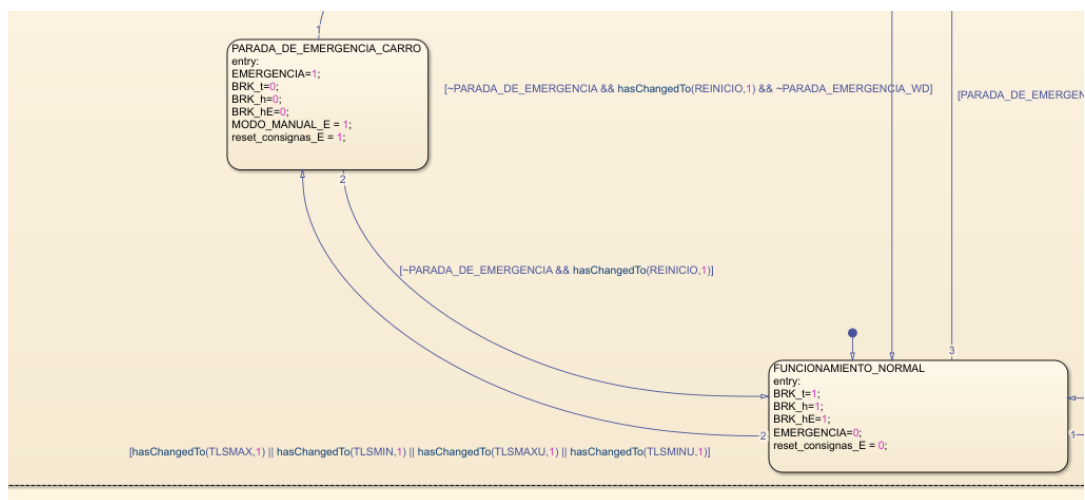


Figura 43: Manejo del estado de emergencia en Simulink (4)

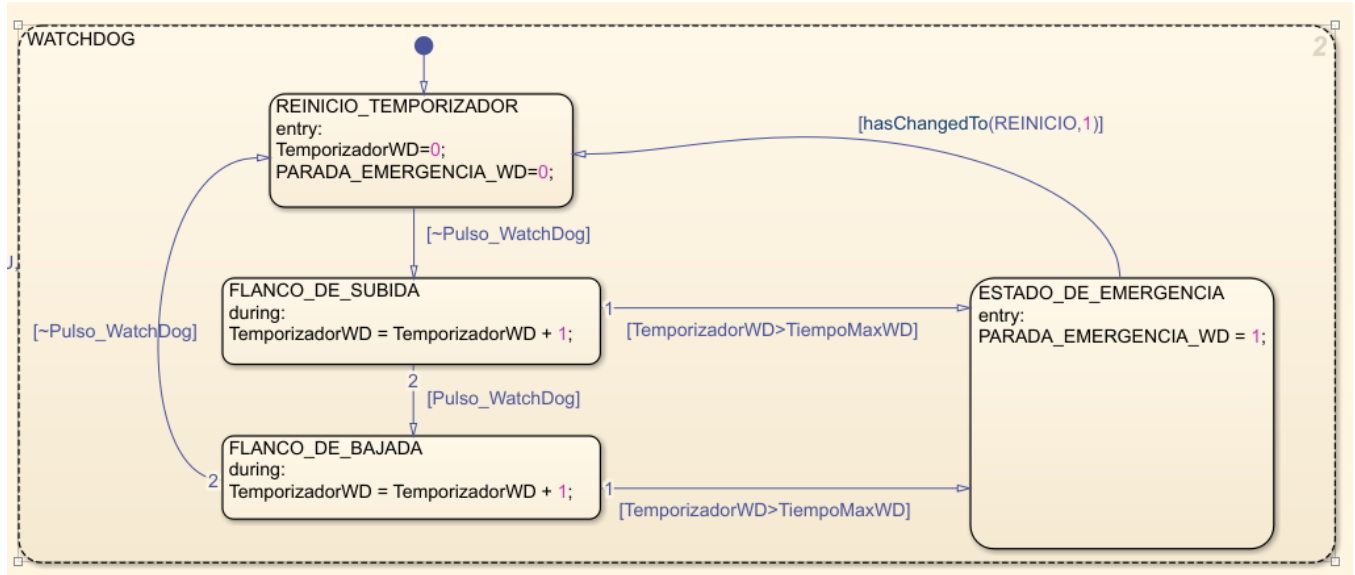


Figura 44: Watchdog control de emergencia en Simulink

En secciones posteriores se implementa en CODESYS, mediante lenguaje SFC una versión análoga del autómata, que es comunicada con Simulink mediante OPC UA.

2.1.6 Interfaz hombre máquina

Con la finalidad de hacer la simulación más amigable para manejo en tiempo real, se implementó una interfaz amigable que representa el estado actual de sistema mediante un esquemático de la grúa junto con testigos luminosos que dan cuenta de diferentes estados posibles.

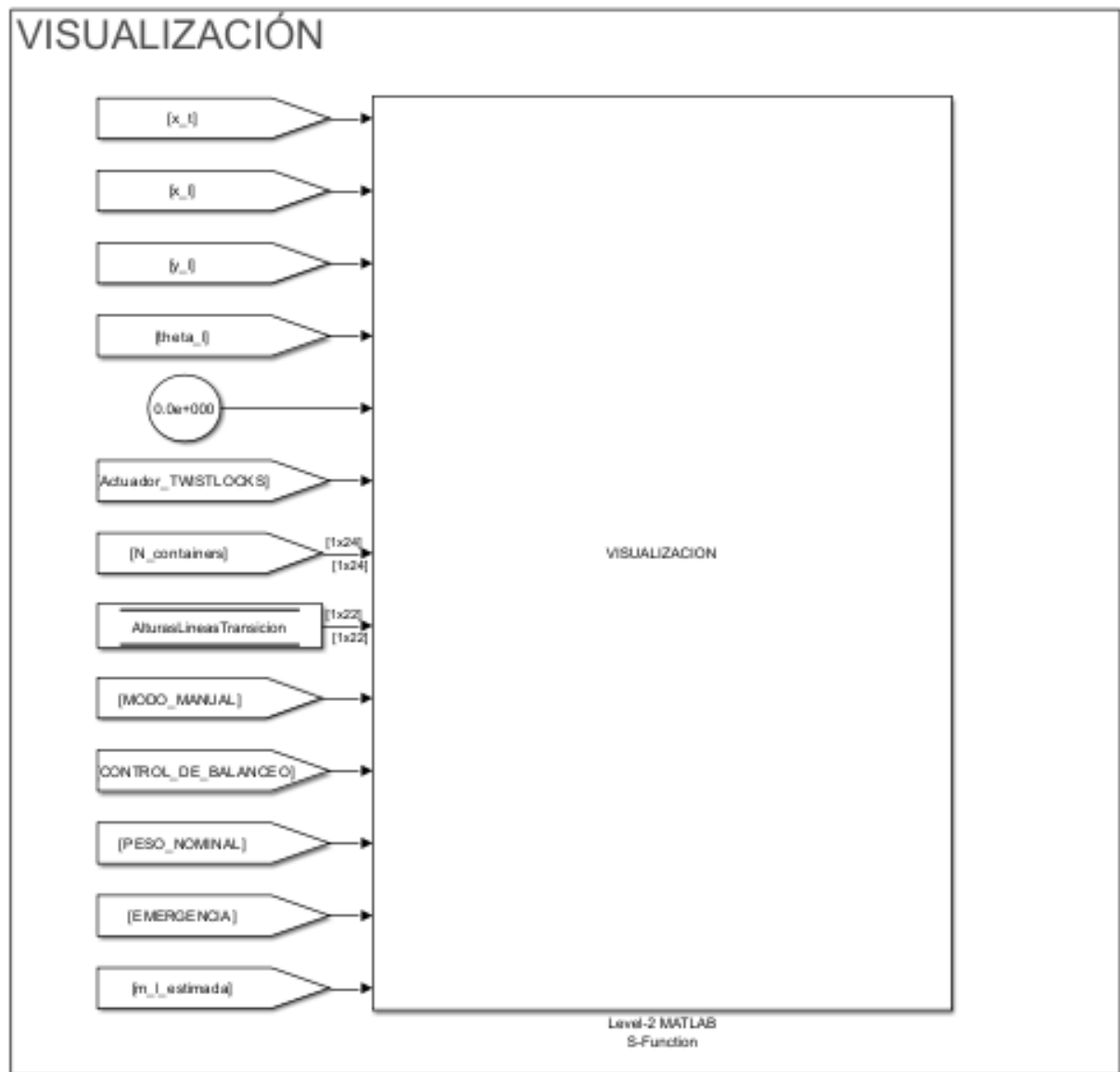


Figura 45: Función de visualización en Simulink

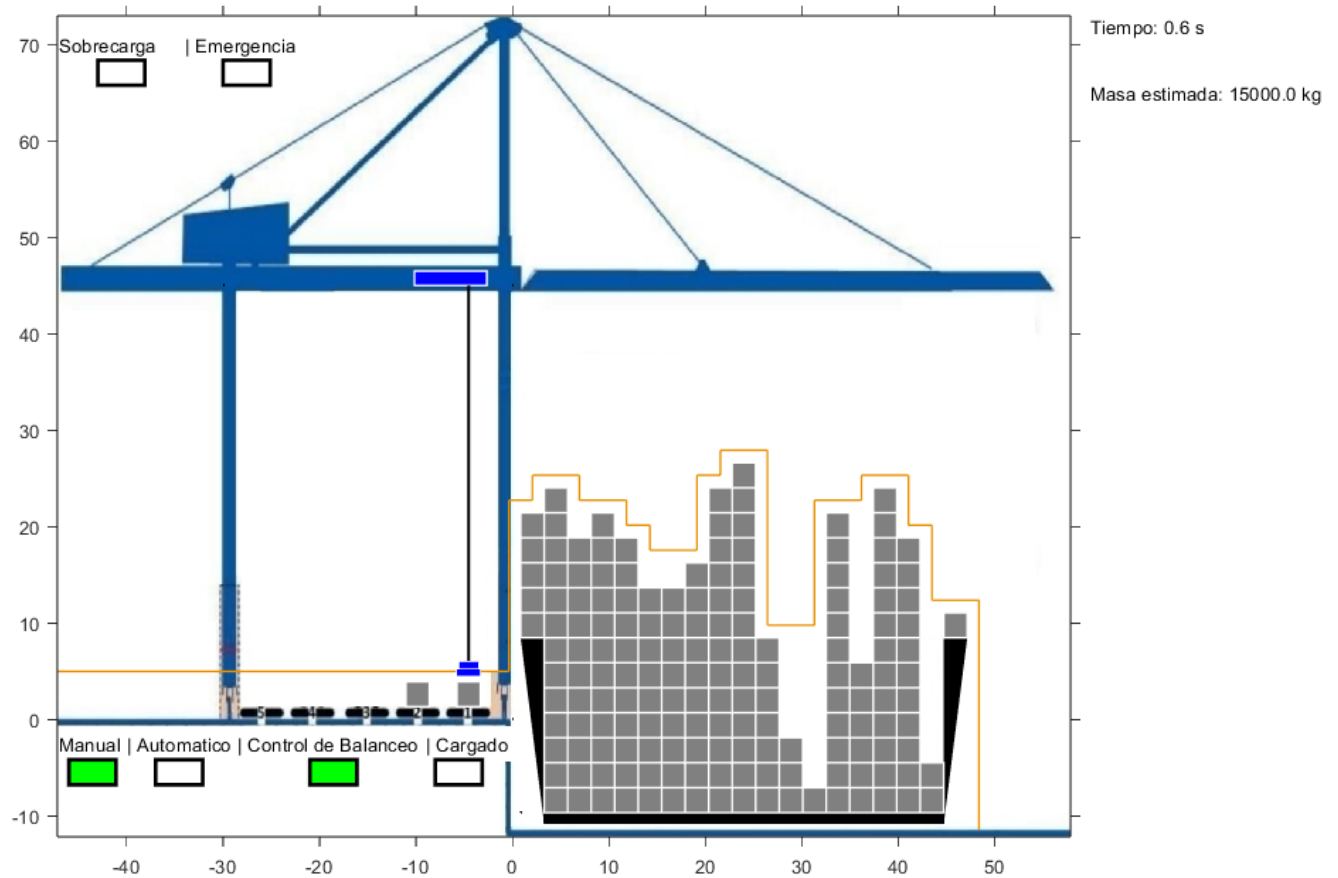


Figura 46: Visualización del sistema en Simulink

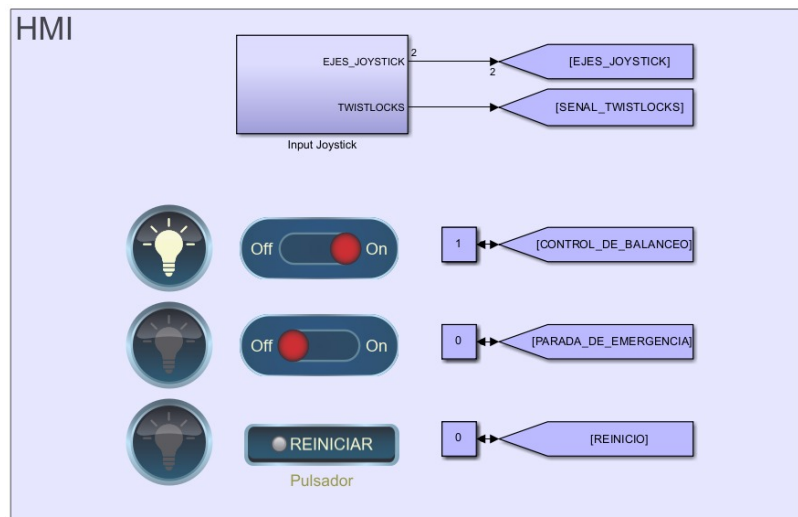


Figura 47: Botones del sistema en Simulink



Figura 48: *Joystick en smartphone*

Además, se utilizó una aplicación que simula un joystick en smartphones.

2.2 SEGUNDA ETAPA, CODESYS Y MATLAB

2.2.1 Implementación del control regulador en bloque de MATLAB

A continuación, según se indica en la sección 1.B, se implementa el control regulador en una función de MATLAB en tiempo discreto. Para una mayor legibilidad, se implementa la acción de control por un lado y la consigna manual por otro. Para la integración se utiliza el método de los trapecios, para ello mediante el uso de bloques de memoria de datos se almacena el estado anterior de algunas relevantes, haciendo que el bloque de función de MATLAB sea análogo a un bloque de función de PLC.

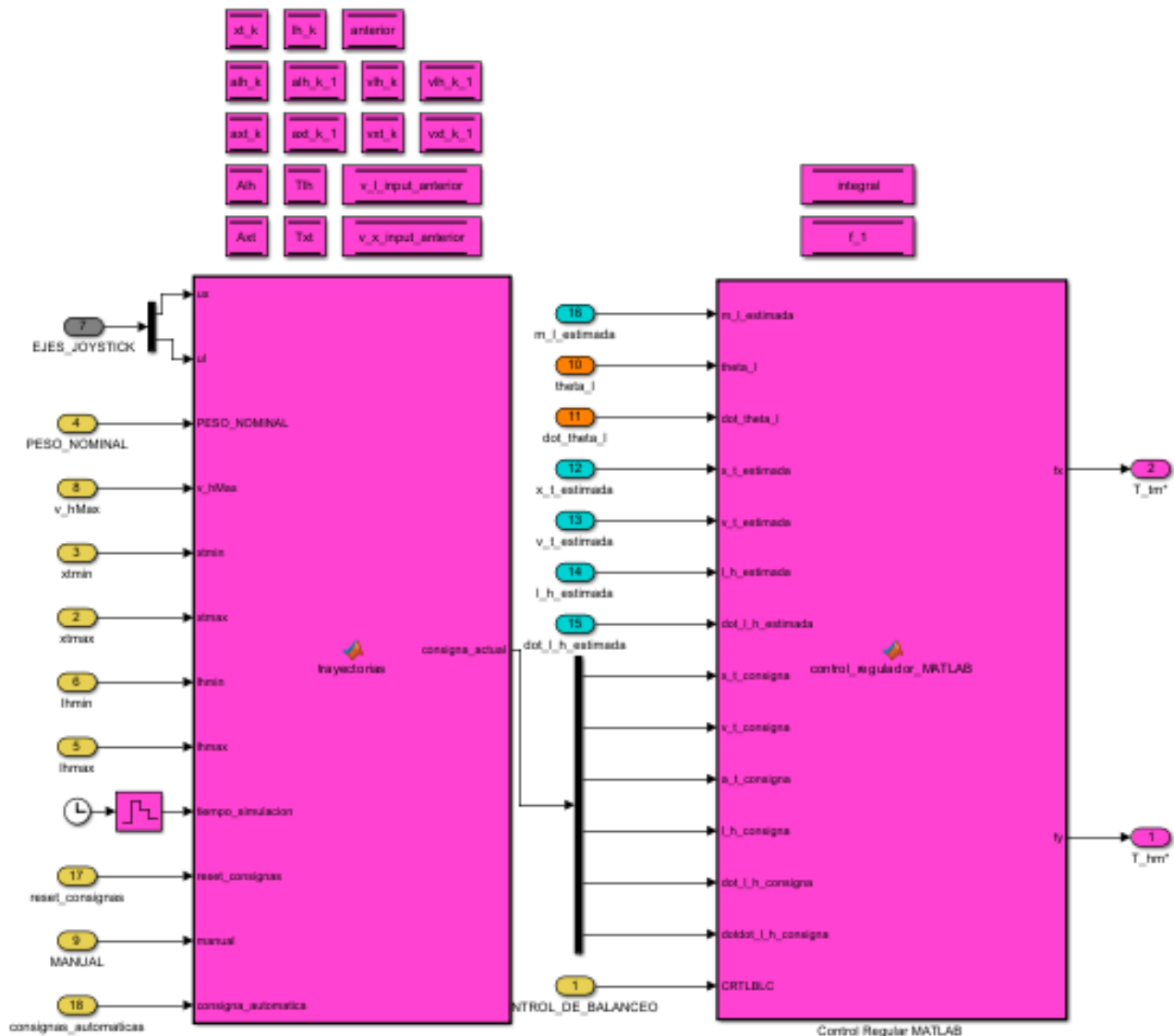


Figura 49: Control regulador en código de MATLAB

Las funciones utilizadas son las siguientes:

```
function [fx,fy] = control_regulador_MATLAB(m_l_estimada, theta_l, dot_theta_l, ...
    M_teqTOT, J_heqTOT, g, Dcc, Kp, Kd, r_td, i_t, r_hd, i_h, x_t_estimada, ...
    v_t_estimada, l_h_estimada, dot_l_h_estimada, x_t_consigna, ...
    v_t_consigna, a_t_consigna, l_h_consigna, dot_l_h_consigna, ...
    dotdot_l_h_consigna, T_s2, zitta, CRTLBLC)

%-----
% Variables que almacenan valores pasados.
%-----
global integral;
global f_1;
%-----
% Cálculo kas.
%-----
if CRTLBLC
    kas = 2*zitta*sqrt(g*l_h_estimada);
else
    kas = 0;
end
%-----
% Cálculo qc y dot_qc.
%-----
qc = [x_t_estimada; l_h_estimada];
dot_qc = [v_t_estimada; dot_l_h_estimada];
%-----
% Cálculo rmc, dot_rmc y dotdot_rmc.
%-----
f_2 = theta_l / cos(theta_l);
integral = integral + T_s2/2 * (f_1 + f_2);
f_1 = f_2;
rmc = [x_t_consigna + kas*integral; l_h_consigna];
dot_rmc = [v_t_consigna + kas*theta_l/cos(theta_l); dot_l_h_consigna];
dotdot_rmc = [a_t_consigna + kas*(1/cos(theta_l) + theta_l*tan(theta_l)/cos(theta_l)) ...
    dot_theta_l; dotdot_l_h_consigna];
%-----
% Cálculo Mcc, Ccc y gc (Dcc constante).
%-----
Mcc = [M_teqTOT + m_l_estimada, m_l_estimada*sin(theta_l); ...
    m_l_estimada*sin(theta_l), J_heqTOT + m_l_estimada];
Ccc = [0, m_l_estimada*cos(theta_l)*dot_theta_l; 0, 0];
gc = [0; -m_l_estimada*g];
%-----
% Cálculo Wcp.
%-----
Wcp = Mcc*dotdot_rmc + Ccc*dot_rmc + Dcc*dot_qc + gc;
%-----
% Cálculo ec y dot_ec.
%-----
ec = rmc - qc;
dot_ec = dot_rmc - dot_qc;
%-----
% Cálculo acción de control.
%-----
F = Wcp + Kp*ec + Kd*dot_ec;
fx = F(1)*r_td/i_t;
fy = -F(2)*r_hd/2/i_h;
end
```

```
function consigna_actual = trayectorias(ux,ul, PESO_NOMINAL,v_tMax, ...
    v_hMax,xtmin,xtmax,lhmin, lhmax, tiempo_simulacion, ...
    reset_consignas, a_hMax, T_s2, manual,a_tMax, ...
    consigna_automatica)

global Alh, Tlh;
global alh_k, vlh_k, lh_k;
global alh_k_1, vlh_k_1;
global v_l_input_anterior;
global Axt, Txt;
global axt_k, vxt_k, xt_k;
global axt_k_1, vxt_k_1;
global v_x_input_anterior;
global anterior;
```

```
% Cálculo de velocidad deseada por el operario.
% Condición de carga nominal.
if PESO_NOMINAL
    if ux > 0
        % Límite en X positivo detectado.
        v_x_input = (1-xtmax)*v_tMax*ux;
    elseif ux < 0
        % Límite en X negativo detectado.
        v_x_input = (1-xtmin)*v_tMax*ux;
    else
        v_x_input = 0;
    end
    if ul > 0
        % Límite en L positivo detectado.
        v_l_input = (1-lhmax)*v_hMax*ul;
    elseif ul < 0
        % Límite en L negativo detectado.
        v_l_input = (1-lhmin)*v_hMax*ul;
    else
        v_l_input = 0;
    end
end
% Condición de carga no nominal.
else
    v_x_input = 0;
    if ul > 0
        v_l_input = 1*v_hMax*ul;
    else
        v_l_input = 0;
    end
end
% Cálculo de aceleración en L actual.
alh_k = interp1(Tlh, Alh, tiempo_simulacion, 'linear');
alh_k(tiempo_simulacion < min(Tlh)) = Alh(1);
alh_k(tiempo_simulacion > max(Tlh)) = Alh(end);
axt_k = interp1(Txt, Axt, tiempo_simulacion, 'linear');
axt_k(tiempo_simulacion < min(Txt)) = Axt(1);
axt_k(tiempo_simulacion > max(Txt)) = Axt(end);
% Recalculo del perfil de aceleración si es necesario.
if v_l_input ~= v_l_input_anterior
    v_l_input_anterior = v_l_input;
    j_lMax = 20;
    delta_v_l = v_l_input - vlh_k;
    bl = sign(delta_v_l);
    tal = (bl*a_hMax - alh_k)/(bl*j_lMax);
    tdl = a_hMax/j_lMax;
    delta_v_jerk_l = alh_k * tal + 0.5 * bl * j_lMax * tal^2;
    delta_v_jerk_total_l = delta_v_jerk_l + bl * a_hMax * tdl - 0.5 * bl * j_lMax * tdl^2;
    if (abs(delta_v_l) > abs(delta_v_jerk_total_l))
        delta_v_constante_l = abs(delta_v_l) - abs(delta_v_jerk_total_l);
        tcl = delta_v_constante_l / a_hMax;
        A_hMax = bl*a_hMax;
    else % Caso triangular
        A_l = bl*j_lMax;
        B_l = 2*alh_k;
        C_l = 0.5*alh_k^2/bl/j_lMax-delta_v_l;
        ta_1_l = (-B_l + sqrt(B_l^2-4*A_l*C_l))/(2*A_l);
        ta_2_l = (-B_l - sqrt(B_l^2-4*A_l*C_l))/(2*A_l);
        if (isreal(ta_1_l) && ta_1_l > 0)
            tal = ta_1_l;
        elseif (isreal(ta_2_l) && ta_2_l > 0)
            tal = ta_2_l;
        end
        tcl = 0;
        tdl = tal + alh_k/bl/j_lMax;
        A_hMax = alh_k + bl*j_lMax*tal;
    end
end
Tl = tal + tcl + tdl;
if isreal(Tl)
    tiempo_perfil_l = linspace(0,Tl,60);
    perfil_aceleracion_l = zeros(1,60);
    esEMC_l = issorted(tiempo_perfil_l, 'strictascend');
    if esEMC_l
        for i=1:60
            if(tiempo_perfil_l(1,i) <= tal)
                perfil_aceleracion_l(1,i) = alh_k + bl*j_lMax*tiempo_perfil_l(1,i);
            elseif(tiempo_perfil_l(i) <= tal+tcl)
                perfil_aceleracion_l(1,i) = A_hMax;
            else
                perfil_aceleracion_l(1,i) = A_hMax - bl*j_lMax*(tiempo_perfil_l(1,i) ...
                    - tal - tcl);
            end
        end
        Alh = perfil_aceleracion_l;
        Tlh = tiempo_perfil_l + tiempo_simulacion;
    end
end
end
```

```

if v_x_input ~= v_x_input_anterior
    v_x_input_anterior = v_x_input;
    j_xMax = 20;
    delta_v_x = v_x_input - vxt_k;
    bx = sign(delta_v_x);
    tax = (bx*a_tMax - axt_k)/(bx*j_xMax);
    tdx = a_tMax/j_xMax;
    delta_v_jerk_x = axt_k * tax + 0.5 * bx * j_xMax * tax^2;
    delta_v_jerk_total_x = delta_v_jerk_x + bx * a_tMax * tdx - 0.5 * bx * j_xMax * tdx^2;
    if (abs(delta_v_x) > abs(delta_v_jerk_total_x))
        delta_v_constante_x = abs(delta_v_x) - abs(delta_v_jerk_total_x);
        tcx = delta_v_constante_x / a_tMax;
        A_xMax = bx*a_tMax;
    else % Caso triangular
        A_x = bx*j_xMax;
        B_x = 2*axt_k;
        C_x = 0.5*axt_k^2/bx/j_xMax-delta_v_x;
        ta_1_x = (-B_x + sqrt(B_x^2-4*A_x*C_x))/(2*A_x);
        ta_2_x = (-B_x - sqrt(B_x^2-4*A_x*C_x))/(2*A_x);

        if (isreal(ta_1_x) && ta_1_x > 0)
            tax = ta_1_x;
        elseif (isreal(ta_2_x) && ta_2_x > 0)
            tax = ta_2_x;
        end
        tcx = 0;
        tdx = tax + axt_k/bx/j_xMax;
        A_xMax = axt_k + bx*j_xMax*tax;
    end
    Tx = tax + tcx + tdx;
    if isreal(Tx)
        tiempo_perfil_x = linspace(0,Tx,60);
        perfil_aceleracion_x = zeros(1,60);
        esEMC_x = issorted(tiempo_perfil_x, 'strictascend');
        if esEMC_x
            for i=1:60
                if(tiempo_perfil_x(1,i) <= tax)
                    perfil_aceleracion_x(1,i) = axt_k + bx*j_xMax*tiempo_perfil_x(1,i);
                elseif(tiempo_perfil_x(i) <= tax+tcx)
                    perfil_aceleracion_x(1,i) = A_xMax;
                else
                    perfil_aceleracion_x(1,i) = A_xMax - bx*j_xMax *(tiempo_perfil_x(1,i) ...
                        - tax - tcx);
                end
            end
            Axt = perfil_aceleracion_x;
            Txt = tiempo_perfil_x + tiempo_simulacion;
        end
    end
end
end

```

```

% Solo trabajamos en estado manual.
if manual
    if anterior
        anterior = manual;
        v_x_input_anterior = 1;
        v_l_input_anterior = 1;
        % Reseteo de integrales.
        lh_k = consigna_automatica(4);
        vlh_k = consigna_automatica(5);
        alh_k = consigna_automatica(6);
        xt_k = consigna_automatica(1);
        vxt_k = consigna_automatica(2);
        axt_k = consigna_automatica(3);
    end
    if reset_consignas
        % en automatico los siguientes valen 0.
        v_x_input_anterior = 1;
        v_l_input_anterior = 1;
        % Reseteo de integrales.
        %lh_k = lh_k;
        vlh_k = 0;
        alh_k = 0;
        Alh = zeros(1,60);
        %xt_k = consigna_automatica(1);
        vxt_k = 0;
        alh_k = 0;
        Axt = zeros(1,60);
    end
end

```

```
% Integración trapezoidal.
vlh_k = vlh_k + T_s2/2*(alh_k + alh_k_1);
lh_k = lh_k + T_s2/2*(vlh_k + vlh_k_1);
vxt_k = vxt_k + T_s2/2*(axt_k + axt_k_1);
xt_k = xt_k + T_s2/2*(vxt_k + vxt_k_1);
% Pasaje de consignas.
    consigna_actual = [xt_k, vxt_k, axt_k, lh_k, vlh_k, alh_k];
else
    consigna_actual = consigna_automatica;
end
% Mantenemos variables actualizadas.
alh_k_1 = alh_k;
vlh_k_1 = vlh_k;
axt_k_1 = axt_k;
vxt_k_1 = vxt_k;
end
```

2.2.2 Implementación del control supervisor en CODESYS

A continuación se implementa el control supervisor en CODESYS mediante programación en SFC, de manera análoga a cómo se hizo en la primera etapa para luego ser conectado a Simulink mediante un esquema servidor-cliente por OPC UA (ver apéndice A), describiéndose cuando se considera oportuno las acciones de entrada, salida o activas de los estados, programadas en ST.

```

23 // Variables I/O Nivel 1.
24 EJES_JOYSTICKS:ARRAY[0..1] OF LREAL;
25 SENAL_TWISTLOCKS:BOOL:=FALSE;
26 yc0xl:LREAL;
27 AlturasLineasTransicion:ARRAY[0..21] OF LREAL;
28 AlturaLineaTransicion:LREAL;
29 x_l_estimada:LREAL;
30 v_lx_estimada:LREAL;
31 y_l_estimada:LREAL;
32 v_ly_estimada:LREAL;
33 v_td_estimada:LREAL;
34 a_td_estimada:LREAL;
35 dot_l_h_estimada:LREAL;
36 dotdot_l_h_estimada:LREAL;
37 consignas:ARRAY[0..5] OF LREAL;
38 F_hw_sensor:LREAL;
39 Reloj:LREAL;
40 MODO_MANUAL:LREAL:=0;
41 Actuador_TWISTLOCKS:LREAL:=0;
42 m_l_estimada:LREAL:=15000;
43 N_containers:ARRAY[0..23] OF LREAL:=[0,0,0,1,1,5,13,11,1];
44 PESO_NOMINAL:LREAL:=1;
45 v_hMax:LREAL:=3;
46 BRK_h:LREAL:=0;
47 BRK_t:LREAL:=0;
48 reset_consignas:LREAL:=0;
49 x_t_consigna_auto:LREAL;
50 v_t_consigna_auto:LREAL;
51 a_t_consigna_auto:LREAL;
52 j_t_consigna_auto:LREAL;
53 l_h_consigna_auto:LREAL;
54 dot_l_h_consigna_auto:LREAL;
55 dotdot_l_h_consigna_auto:LREAL;
56 dotdotdot_l_h_consigna_auto:LREAL;
57 Pulso_WatchDog_Nivel1:LREAL;
58 END VAR

```

Figura 50: Definición de las variables de entrada/salida del control supervisor en CODESYS

```

1  PROGRAM PLC_PRG_SUP
2  VAR
3      // Variables internas del control supervisor.
4      permitir_cambio:BOOL;
5      distancia:LREAL;
6      ind:INT;
7      EJES_JOYSTICK_anterior:ARRAY[0..1] OF LREAL;
8      j_h_consigna_auto:LREAL;
9      TiempoInicialReloj_Izaje:LREAL;
10     y_h_consigna_auto:LREAL;
11     v_h_consigna_auto:LREAL;
12     a_h_consigna_auto:LREAL;
13     x_t_Final:LREAL;
14     y_h_Final:LREAL;
15     TiempoInicialReloj_Translacion:LREAL;
16     signo:LREAL;
17     TIEMPOS_t:LREAL;
18     delta_pos_total_t:LREAL;
19     delta_t_total_t:LREAL;
20     TIEMPOS_h:LREAL;
21     delta_t_total_h:LREAL;
22     delta_pos_total_h:LREAL;
23     TiempoCambioModoManual_a_Automatico:LREAL;
24     Descenso_Manual:BOOL;
25     Actuador_TWISTLOCKS_prev:LREAL;
26     TraslacionCompletada:BOOL;
27     IzajeCompletado:BOOL;
28     IzajeDescensoCompletado:BOOL;
29     i_sec:INT:=-1;
30     index_menor:INT;
31     index_x_l_estimada:INT;
32     index_mayor:INT;
33     index_x_t_Final:INT;
34     H:LREAL;
35     ValidacionIzajeDesacelerando:BOOL;
36     inhibir:BOOL;
37     ValidacionCarroDesacelerando:BOOL;
38     W_c:LREAL:=2.4340;
39     M_cabezal:LREAL:=15000;

```

Figura 51: Definición de las variables de internas del control supervisor en CODESYS

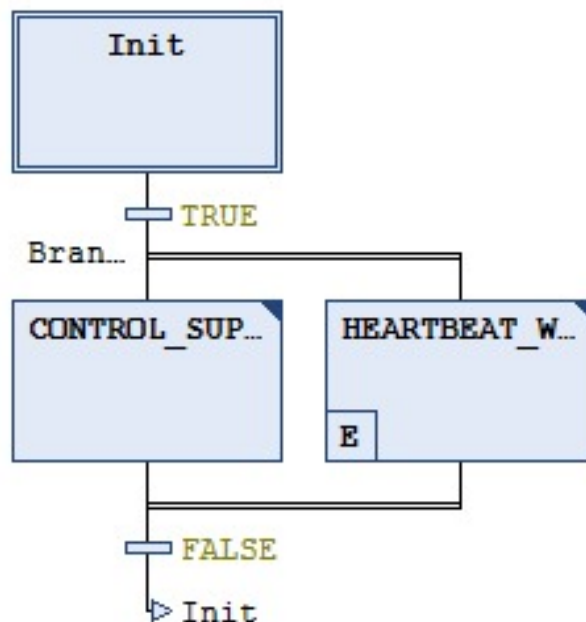


Figura 52: Control supervisor en SFC CODESYS (1)

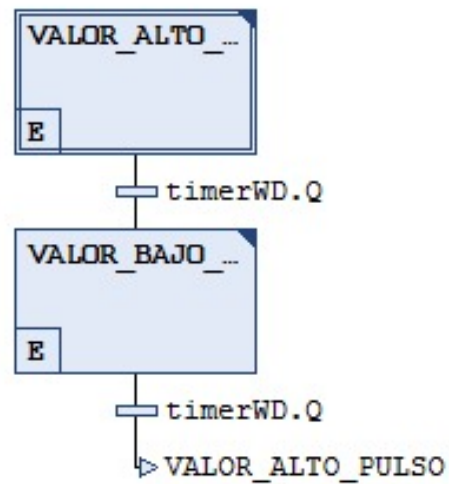


Figura 53: Control supervisor en SFC CODESYS (2)

```

1 | timerWD(IN:=TRUE,PT:=INT_TO_TIME(250));
  
```

Figura 54: VALOR_ALTO_PULSO acción activa

```

1 | GVL.Pulso_WatchDog_Nivell:=1;
2 | timerWD(IN:=FALSE);
  
```

Figura 55: VALOR_ALTO_PULSO acción entrada

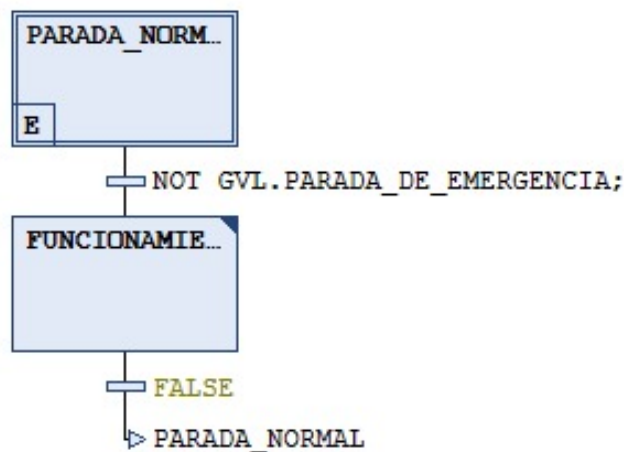


Figura 56: Control supervisor en SFC CODESYS (3)

```

1 // Inicialización estado de la carga.
2 GVL.m_l_estimada := M_cabezal;
3 //N_containers := n_containers; // Ya inicializado.
4 GVL.v_hMax := v_hMax_Unloaded;
5 GVL.PESO_NOMINAL := 1;
6 GVL.Actuador_TWISTLOCKS := 0;
7 permitir_cambio := FALSE;
8 inhibir := FALSE;
9 //Inicialización MODO MANUAL:
10 GVL.MODO_MANUAL := 1;
11 H := 0; //Toma el valor de H_C o 0 [m] según si se tiene o no un container).
12 //Inicialización MODO AUTOMÁTICO:
13 i_sec := 0;
14 Descenso_Manual := FALSE;
15 TraslacionCompletada := FALSE;
16 IzajeCompletado := FALSE;
17 IzajeDescensoCompletado := FALSE;
18 //Inicialización FRENOS DE OPERACIÓN:
19 GVL.v_t_consigna_auto := 0;
20 GVL.dot_l_h_consigna_auto := 0;
21 ValidacionCarroDesacelerando := TRUE;
22 ValidacionIzajeDesacelerando := TRUE;
23 GVL.BRK_t := 1; //0;
24 GVL.BRK_h := 1; //0;
25 ///////////////////////////////////////////////////
26 GVL.reset_consignas := 0;
27 GVL.Pulso_WatchDog_Nivel1 := 0;

```

Figura 57: PARADA_NORMAL acción entrada

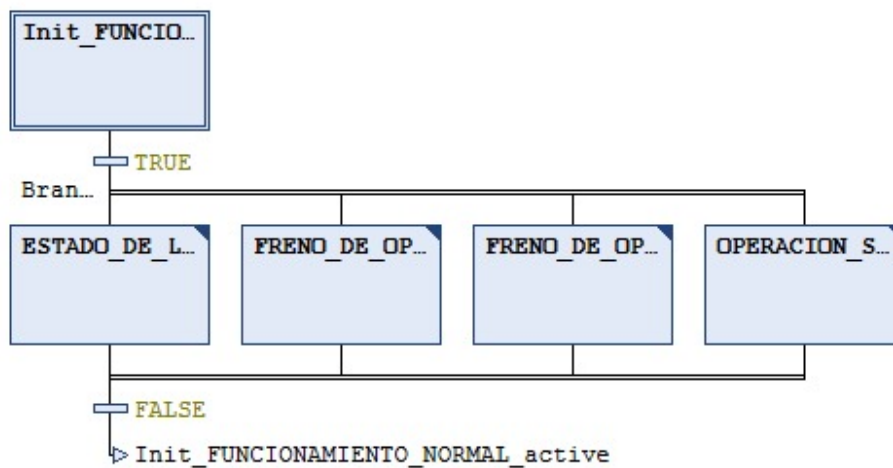


Figura 58: Control supervisor en SFC CODESYS (4)

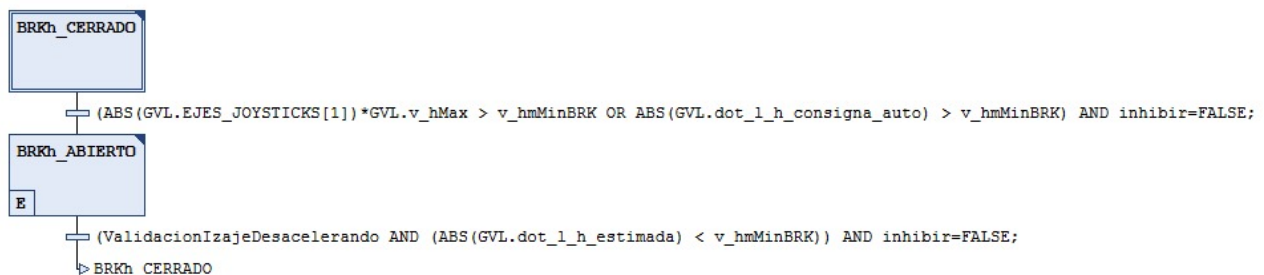


Figura 59: Control supervisor en SFC CODESYS (5)

```

1 IF (GVL.dot_l_h_estimada * GVL.dotdot_l_h_estimada < 0) THEN
2   ValidacionIzajeDesacelerando := TRUE;
3 ELSE
4   ValidacionIzajeDesacelerando := FALSE;
5 END_IF

```

Figura 60: BRKh_ABIERTO acción activa

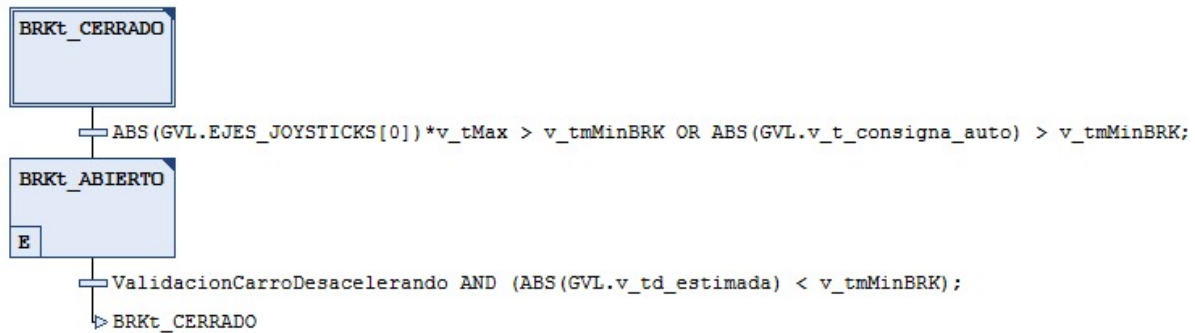


Figura 61: Control supervisor en SFC CODESYS (6)

```

1 IF ( GVL.v_td_estimada * GVL.a_td_estimada < 0) THEN
2   ValidacionCarroDesacelerando := TRUE;
3 ELSE
4   ValidacionCarroDesacelerando := FALSE;
5 END_IF

```

Figura 62: BRKt_ABIERTO acción activa

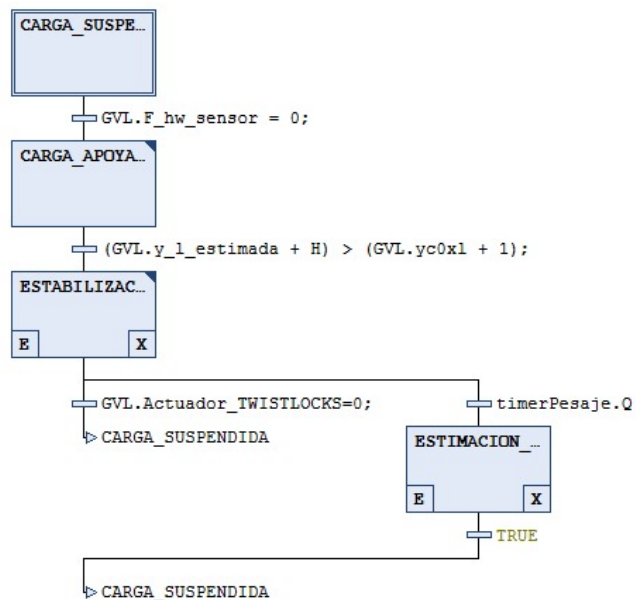


Figura 63: Control supervisor en SFC CODESYS (7)

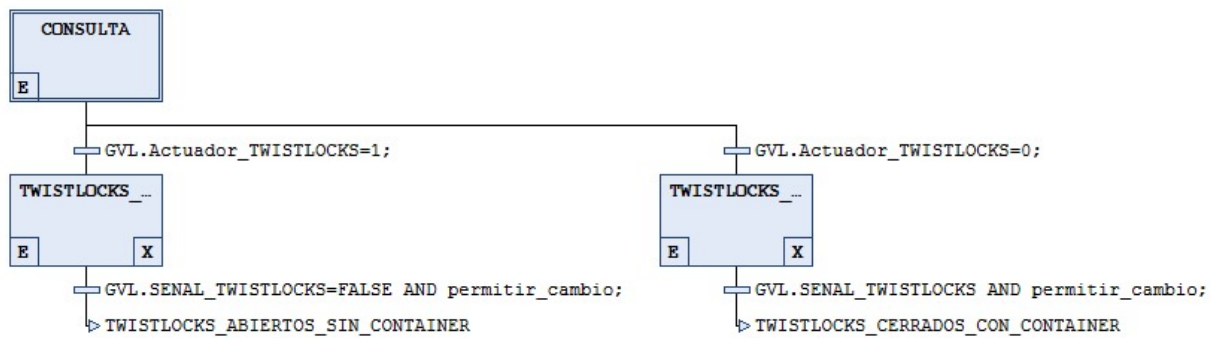


Figura 64: Control supervisor en SFC CODESYS (8)

```

1  GVL.m_l_estimada:=M_cabecal;
2  GVL.Actuador_TWISTLOCKS:=1;
3  // Comprobación de la ubicación para dejar container:
4  retornoFuncion:=ValorMasCercanoConIndice(X_cc:=X_cc,x_l_estimada:=GVL.x_l_estimada,distancia=>distancia,ind=>ind);
5  IF distancia < 0.4*W_c THEN// Comprobamos estar cerca.
6      permitir_cambio := TRUE;
7  ELSE
8      permitir_cambio := FALSE;
9  END IF
10 inhibir := TRUE;
11

```

Figura 65: TWISTLOCKS_CERRADOS_CON_CONTAINER acción entrada

```

1  GVL.N_containers[ind]:=GVL.N_containers[ind]+1;

```

Figura 66: TWISTLOCKS_CERRADOS_CON_CONTAINER acción salida

```

1  GVL.m_l_estimada:=M_cabecal;
2  GVL.Actuador_TWISTLOCKS:=0;
3  // Comprobación de la ubicación para tomar container:
4  retornoFuncion:=ValorMasCercanoConIndice(X_cc:=X_cc,x_l_estimada:=GVL.x_l_estimada,distancia=>distancia,ind=>ind);
5  IF distancia < 0.4*W_c THEN// Comprobamos estar cerca.
6      IF GVL.N_containers[ind] > 0 THEN
7          permitir_cambio := TRUE;
8      END IF
9  ELSE
10     permitir_cambio := FALSE;
11 END IF
12 GVL.v_hMax := v_hMax_Unloaded;
13 inhibir := FALSE;
14 GVL.PESO_NOMINAL:=1;

```

Figura 67: TWISTLOCKS_CERRADOS_SIN_CONTAINER acción entrada

```

1  GVL.N_containers[ind]:=GVL.N_containers[ind]-1;

```

Figura 68: TWISTLOCKS_CERRADOS_SIN_CONTAINER acción salida

```

1  timerPesaje(IN:=TRUE,PT:=INT_TO_TIME(4000));
2  GVL.BRK_h:=0;

```

Figura 69: ESTABILIZACION_CARGA acción activa

```

1 timerPesaje(IN:=FALSE);
2 IF GVL.Actuador_TWISTLOCKS = 0 THEN
3   GVL.m_l_estimada := M_cabecal;
4   GVL.PESO_NOMINAL := 1;
5   GVL.v_hMax := v_hMax_Unloaded;
6 ELSE
7   GVL.v_hMax := 0;
8   GVL.BRK_h := 0;
9 END_IF

```

Figura 70: ESTABILIZACION_CARGA acción entrada

```

1 inhibir:=FALSE;

```

Figura 71: ESTABILIZACION_CARGA acción salida

```

1 GVL.reset_consignas := 1;
2 GVL.m_l_estimada := GVL.F_hw_sensor*2/g;
3 IF GVL.m_l_estimada >= 65000 THEN
4   GVL.PESO_NOMINAL := 0;
5   GVL.v_hMax := 1;
6 ELSIF GVL.m_l_estimada >= 32500 THEN
7   GVL.PESO_NOMINAL := 1;
8   GVL.v_hMax := 956150/(GVL.m_l_estimada*g);
9 ELSE // Carga menor a 32500 kg:
10  GVL.PESO_NOMINAL := 1;
11  GVL.v_hMax := v_hMax_Unloaded;
12 END_IF

```

Figura 72: ESTIMACION_CARGA acción entrada

```

1 GVL.reset_consignas:=0;

```

Figura 73: ESTIMACION_CARGA acción salida

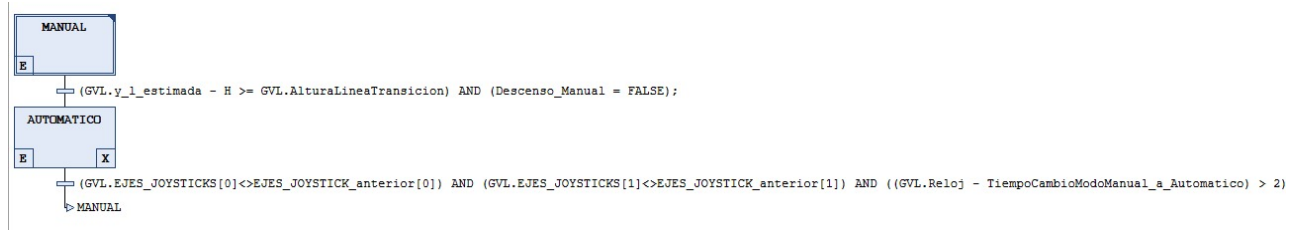


Figura 74: Control supervisor en SFC CODESYS (9)

```

1 GVL.MODO_MANUAL:=1;
2 Actuador_TWISTLOCKS_prev := GVL.Actuador_TWISTLOCKS;

```

Figura 75: MANUAL acción entrada

```

1  IF (Actuador_TWISTLOCKS_prev <> GVL.Actuador_TWISTLOCKS) THEN
2      Descenso_Manual := NOT Descenso_Manual;
3      TraslacionCompletada := FALSE;
4      IF (GVL.Actuador_TWISTLOCKS = 0) THEN
5          H := 0;
6      ELSE
7          H := H_c;
8      END_IF
9  END_IF

```

Figura 76: MANUAL acción activa

```

1  // Condiciones Iniciales:
2  GVL.x_t_consigna_auto := GVL.x_l_estimada; // x_inicial
3  GVL.v_t_consigna_auto := GVL.v_lx_estimada; // v_inicial
4  GVL.a_t_consigna_auto := 0; // a_inicial
5  y_h_consigna_auto := GVL.y_l_estimada; // y_inicial
6  v_h_consigna_auto := GVL.v_ly_estimada; // v_inicial
7  a_h_consigna_auto := 0; // a_inicial
8  GVL.j_t_consigna_auto := 0;
9  IF (ABS(GVL.v_t_consigna_auto) < 0.1) THEN
10     GVL.v_t_consigna_auto := 0;
11 END_IF
12 IF (ABS(v_h_consigna_auto) < 0.1) THEN
13     v_h_consigna_auto := 0;
14 END_IF
15 GVL.l_h_consigna_auto := Y_t0 - y_h_consigna_auto;
16 GVL.dot_l_h_consigna_auto := -v_h_consigna_auto;
17 GVL.dotdot_l_h_consigna_auto := -a_h_consigna_auto;
18 GVL.dotdotdot_l_h_consigna_auto := 0;
19 //////////////////////////////////////////////////
20 GVL.MODO_MANUAL:=0;
21 i_sec := i_sec + 1;
22 TiempoCambioModoManual_a_Automatico:=GVL.Reloj;
23 x_t_Final := SecuenciaDeTrabajoX[i_sec];
24 index_x_l_estimada := convertir_X_a_index(pos_x:=GVL.x_l_estimada,PosicionesX_LineasTransicion:=PosicionesX_LineasTransicion,W_c:=W_c);
25 index_x_t_Final := convertir_X_a_index(pos_x:=x_t_Final,PosicionesX_LineasTransicion:=PosicionesX_LineasTransicion,W_c:=W_c);
26 index_menor := MIN(index_x_l_estimada, index_x_t_Final);
27 index_mayor := MAX(index_x_l_estimada, index_x_t_Final);

```

Figura 77: AUTOMATICO acción entrada (1)

```

28 // Inicializamos primer valor.
29 maxVal := GVL.AlturasLineasTransicion[index_menor];
30 // Buscamos el máximo.
31 FOR indiceMaximo := index_menor + 1 TO index_mayor DO
32     IF GVL.AlturasLineasTransicion[indiceMaximo] > maxVal THEN
33         maxVal := GVL.AlturasLineasTransicion[indiceMaximo];
34     END_IF
35 END_FOR;
36 y_h_Final := maxVal + H;
37 // Condicional simplemente para corregir y_h_Final en el descenso:
38 IF ( index_x_t_Final > 1 AND (GVL.AlturasLineasTransicion[index_x_t_Final-1] > GVL.AlturasLineasTransicion[index_x_t_Final]) ) THEN
39     index_x_t_Final := index_x_t_Final - 1;
40 END_IF
41 TraslacionCompletada := FALSE;
42 IzajeCompletado := FALSE;
43 IzajeDescensoCompletado := FALSE;
44 EJES_JOYSTICK_anterior := GVL.EJES_JOYSTICKS;
45

```

Figura 78: AUTOMATICO acción entrada (2)

```

1  FUNCTION_BLOCK obtenerTiemposPerfilATrapezoidal
2  VAR_INPUT
3      pos_inicial: LREAL;
4      vel_inicial: LREAL;
5      acel_inicial: LREAL;
6      pos_final: LREAL;
7      VelMax: LREAL;
8      AcelMax: LREAL;
9      JerkMax: LREAL;
10 END_VAR
11 VAR_OUTPUT
12     TIEMPOS: ARRAY [0..6] OF LREAL;
13     delta_t_total: LREAL;
14     delta_pos_total: LREAL;
15 END_VAR
16 VAR_TEMP
17     vel_inicial_0: LREAL;
18     acel_inicial_0: LREAL;
19     abs_delta_pos_total: LREAL;
20     delta_t_j5: LREAL;
21     delta_v_desacel_const: LREAL;
22     delta_t_j1: LREAL;
23     delta_t_a2: LREAL;
24     delta_t_j3: LREAL;
25     delta_t_v4: LREAL;
26     delta_t_a6: LREAL;
27     a_1: LREAL;
28     v1: LREAL;
29     delta_x1: LREAL;
30     delta_x123: LREAL;
31     v5: LREAL;

```

Figura 79: Función obtenerTiemposATrapezoidal en ST (1)

```

31     v5: LREAL;
32     delta_x567: LREAL;
33     k: LREAL;
34     limite_inferior: LREAL;
35     limite_superior: LREAL;
36     V_peak: LREAL;
37     error: LREAL;
38     iter: DINT;
39     a_1_tmp: LREAL;
40     delta_t_a6_tmp: LREAL;
41     delta_v5_tmp: LREAL;
42     delta_v5_tmp_0: LREAL;
43     exitgl: DINT;
44 END_VAR
45

```

Figura 80: Función obtenerTiemposATrapezoidal en ST (2)

```

32 END_IF;
33 delta_t_j3 := a_1 / JerkMax;
34 // Tiempo para reducir aceleración a 0.
35 // Velocidad después de fase 1 de jerk positivo:
36 a_1_tmp := delta_t_j1 * delta_t_j1;
37 v1 := ((acel_inicial_0 * delta_t_j1) + vel_inicial_0) + ((0.5 * JerkMax) * a_1_tmp);
38 // Ganancia de Velocidad (Variación) en la fase 3 de jerk negativo:
39 // Tiempo de aceleración constante:
40 IF a_1 > 0.0 THEN
41     delta_t_a2 := ((VelMax - v1) - ((0.5 * JerkMax) * (delta_t_j3 * delta_t_j3))) / a_1;
42     // Delta_t= Delta_V / A_const
43 ELSE
44     delta_t_a2 := 0.0;
45 END_IF
46 // Cálculo de distancias: *)
47 delta_x1 := (((0.5 * acel_inicial_0) * a_1_tmp) + (vel_inicial_0 * delta_t_j1)) + ((0.1666666666666666 * JerkMax) * EXPDT(delta_t_j1, 3.0));
48 vel_inicial_0 := (0.5 * AcelMax) * (delta_t_j3 * delta_t_j3);
49 acel_inicial_0 := (0.1666666666666666 * JerkMax) * EXPDT(delta_t_j3, 3.0);
50 delta_x123 := (((AcelMax * delta_t_a2) + v1) * delta_t_j3) + vel_inicial_0 - acel_inicial_0 + (((0.5 * AcelMax) * (delta_t_a2 * delta_t_a2)) + (v1 * delta_t_a2))
51 // Tiempos para desaceleración:
52 // Velocidad después de fase 5 de jerk negativo:
53 delta_t_a6_tmp := VelMax - delta_v5_tmp_0;
54 delta_t_a6 := (delta_t_a6_tmp - delta_v5_tmp_0) / AcelMax;
55 a_1_tmp := (0.1666666666666666 * JerkMax) * EXPDT(delta_t_j5, 3.0);
56 delta_v5_tmp := (0.5 * AcelMax) * delta_t_a6;
57 delta_x567 := (((((-AcelMax) * delta_t_a6) + delta_t_a6_tmp) * delta_t_j5) - delta_v5_tmp) + a_1_tmp + (((VelMax * delta_t_j5) - a_1_tmp) + ((delta_t_a6_tmp * delt
58 IF (delta_x123 + delta_x567) <= abs_delta_pos_total THEN
59     delta_t_v4 := ((abs_delta_pos_total - delta_x123) - delta_x567) / VelMax;
60     // Tiempo velocidad constante
61 ELSE

```

Figura 81: Función obtenerTiemposATrapezoidal en ST (3)

```

62 // No hay fase de velocidad constante:
63 delta_t_v4 := 0.0;
64 // Se usa un método de BÚSQUEDA BINARIA para encontrar V_peak:
65 delta_t_a6_tmp := AcelMax * AcelMax;
66 k := (delta_t_a6_tmp * 0.5) / JerkMax;
67 // k es un parámetro auxiliar.
68 limite_inferior := v1;
69 // Límite inferior para V_peak
70 limite_superior := VelMax;
71 // Límite superior para V_peak *)
72 iter := 0;
73 REPEAT
74     exitgl := 0;
75     IF iter < 100 THEN
76         V_peak := (limite_inferior + limite_superior) / 2.0;
77         // Calcular nuevos tiempos de aceleración constante en función de V_peak: *)
78         delta_t_a2 := MAX(((V_peak - v1) - k) / AcelMax, 0.0);
79         delta_t_a6 := MAX((V_peak - (delta_t_a6_tmp / JerkMax)) / AcelMax, 0.0);
80         // Recalcular distancias con V_peak:
81         v5 := V_peak - delta_v5_tmp_0;
82         error := (((((AcelMax * delta_t_a2) + v1) * delta_t_j3) + vel_inicial_0) - acel_inicial_0) + (((0.5 * AcelMax) * (delta_t_a2 * delta_t_a2)) + (v1 * del
83         IF ABS(error) < 0.001 THEN
84             exitgl := 1;
85         ELSE
86             IF error > 0.0 THEN
87                 limite_superior := V_peak;
88             ELSE
89                 limite_inferior := V_peak;
90             END IF
91             iter := iter + 1;
92     END IF

```

Figura 82: Función obtenerTiemposATrapezoidal en ST (4)

```

92     END_IF
93 ELSE
94     exitgl := 1;
95 END_IF
96 UNTIL NOT(exitgl = 0)
97 END_REPEAT
98 END_IF
99 // Tiempo total:
100 TIEMPOS[0] := delta_t_j1;
101 TIEMPOS[1] := delta_t_a2;
102 TIEMPOS[2] := delta_t_j3;
103 TIEMPOS[3] := delta_t_v4;
104 TIEMPOS[4] := delta_t_j5;
105 TIEMPOS[5] := delta_t_a6;
106 TIEMPOS[6] := delta_t_j5;
107
108 delta_t_total := (((((delta_t_j1 + delta_t_a2) + delta_t_j3) + delta_t_v4) + delta_t_j5) + delta_t_a6) + delta_t_j5;
109
110

```

Figura 83: Función obtenerTiemposATrapezoidal en ST (5)

```

1  FUNCTION_BLOCK moverPorTrayectoriaAutomatica
2  VAR_INPUT
3      TIEMPOS: ARRAY[0..5] OF LREAL;
4      delta_t_total: LREAL;
5      delta_pos_total: LREAL;
6      pos_anterior_: LREAL;
7      vel_anterior_: LREAL;
8      acel_anterior_: LREAL;
9      JerkMax: LREAL;
10     tiempoInicialReloj: LREAL;
11     Reloj:LREAL;
12 END_VAR
13 VAR_OUTPUT
14     posicion_: LREAL;
15     velocidad_: LREAL;
16     aceleracion_: LREAL;
17     jerk_: LREAL;
18 END_VAR
19 VAR_TEMP
20     TIEMPOS_ACUM:ARRAY[0..5] OF LREAL;
21     TiempoInicioMovimiento:LREAL;
22     vel_anteriorZZ0: LREAL;
23     acel_anteriorZZ0: LREAL;
24     signo: DINT;
25     tmp: LREAL;
26     T_sl:LREAL:=0.02;
27 END_VAR
28
29

```

Figura 84: Función moverPorTrayectoriaAutomatica en ST (1)

```

1  TIEMPOS_ACUM[0] := TIEMPOS[0];
2  TIEMPOS_ACUM[1] := TIEMPOS[0] + TIEMPOS[1];
3  TIEMPOS_ACUM[2] := TIEMPOS[0] + TIEMPOS[1] + TIEMPOS[2];
4  TIEMPOS_ACUM[3] := TIEMPOS[0] + TIEMPOS[1] + TIEMPOS[2] + TIEMPOS[3];
5  TIEMPOS_ACUM[4] := TIEMPOS[0] + TIEMPOS[1] + TIEMPOS[2] + TIEMPOS[3] + TIEMPOS[4];
6  TIEMPOS_ACUM[5] := TIEMPOS[0] + TIEMPOS[1] + TIEMPOS[2] + TIEMPOS[3] + TIEMPOS[4] + TIEMPOS[5]; // Sumas acumulativas de TIEMPOS
7  IF delta_pos_total > 0.0 THEN
8      signo := 1;
9  ELSIF delta_pos_total < 0.0 THEN
10     signo := -1;
11 ELSE
12     signo := 0;
13 END_IF
14 tmp:=0.038;
15 TiempoInicioMovimiento:=tiempoInicialReloj+tmp;
16
17 // Generación del perfil. Determinar fase actual:
18 IF ((TiempoInicioMovimiento < Reloj) AND (Reloj < TIEMPOS_ACUM[0] + TiempoInicioMovimiento)) THEN
19     jerk_ := signo*JerkMax; // FASE 1: Jerk positivo (incremento de aceleración positiva).
20 ELSIF Reloj < TIEMPOS_ACUM[1] + TiempoInicioMovimiento THEN
21     jerk_ := 0; // FASE 2: Aceleración positiva constante.
22 ELSIF Reloj < TIEMPOS_ACUM[2] + TiempoInicioMovimiento THEN
23     jerk_ := -signo*JerkMax; // FASE 3: Jerk negativo (disminución de aceleración positiva a cero).
24 ELSIF Reloj < TIEMPOS_ACUM[3] + TiempoInicioMovimiento THEN
25     jerk_ := 0; // FASE 4: Velocidad constante.
26 ELSIF Reloj < TIEMPOS_ACUM[4] + TiempoInicioMovimiento THEN
27     jerk_ := -signo*JerkMax; // FASE 5: Jerk negativo (incremento de aceleración negativa).
28 ELSIF Reloj < TIEMPOS_ACUM[5] + TiempoInicioMovimiento THEN
29     jerk_ := 0; // FASE 6: Aceleración negativa constante.
30 ELSIF Reloj < delta_t_total + TiempoInicioMovimiento THEN
31     jerk_ := signo*JerkMax; // FASE 7: Jerk positivo (disminución de aceleración negativa a cero).
32 ELSE
33     jerk_ := 0;
34 //Ajustar condiciones finales exactas:
35     acel_anterior_ := 0;
36     vel_anterior_ := 0;
37 END_IF

```

Figura 85: Función moverPorTrayectoriaAutomatica en ST (2)

```

39 // Actualizar aceleración con límites:
40 aceleracion_ := acel_anterior + jerk_ * T_sl; //aceleracion = acel_anterior + jerk_ * T_sl;
41
42 // Actualizar velocidad con límites:
43 velocidad_ := vel_anterior + aceleracion_ * T_sl;
44
45 // Actualizar posición:
46 posicion_ := pos_anterior + velocidad_ * T_sl;

```

Figura 86: Función moverPorTrayectoriaAutomatica en ST (3)

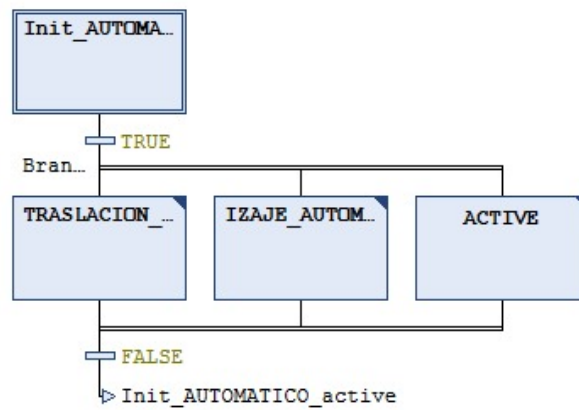


Figura 87: Control supervisor en SFC CODESYS (10)

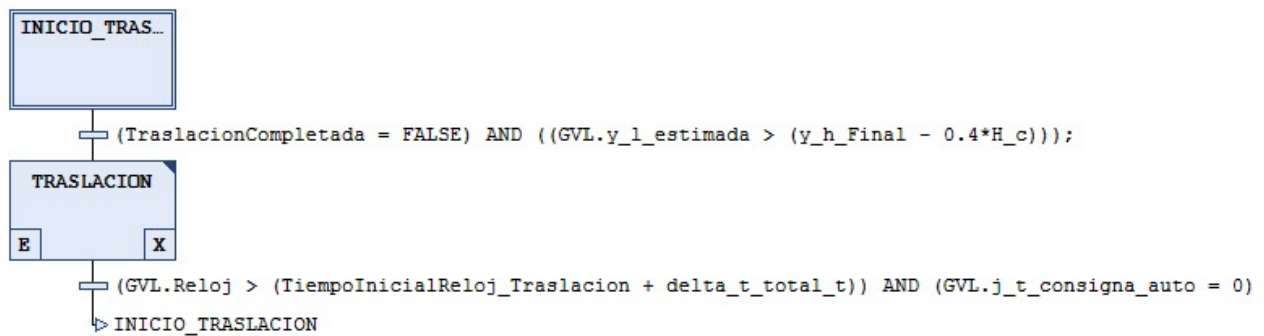


Figura 88: Control supervisor en SFC CODESYS (11)

```

1 // Condiciones Iniciales:
2 GVL.x_t_consigna_auto := GVL.consignas[0];
3 GVL.v_t_consigna_auto := GVL.consignas[1];
4 GVL.a_t_consigna_auto := 0;
5 IF (ABS(GVL.v_t_consigna_auto) < 0.01 ) THEN
6     GVL.v_t_consigna_auto := 0;
7 END_IF
8 TiempoInicialReloj_Traslacion:=GVL.Relej;
9 //Trayectoria Horizontal:
10 obtenerTiemposPerfilATrapezoidalCalculo(pos_inicial:=GVL.x_t_consigna_auto,vel_inicial:=GVL.v_t_consigna_auto,acel_inicial:=GVL.a_t_consigna_auto,pos_final:=x_t_final,VelMax:=

```

Figura 89: TRASLACION acción entrada

```
1 InicialReloj_Traslacion,Relej:=GVL.Relej,posicion=>GVL.x_t_consigna_auto,velocidad=>GVL.v_t_consigna_auto,aceleracion=>GVL.a_t_consigna_auto,jerk=>GVL.j_t_consigna_auto);
```

Figura 90: *TRASLACION acción activa*

```
1 TraslacionCompletada := TRUE;
2
```

Figura 91: *TRASLACION acción salda*

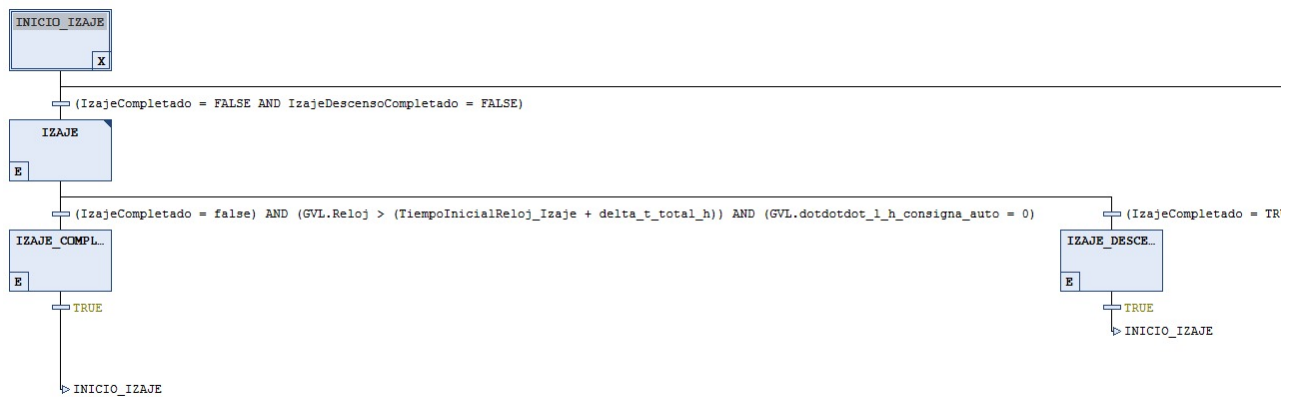


Figura 92: *Control supervisor en SFC CODESYS (12)*

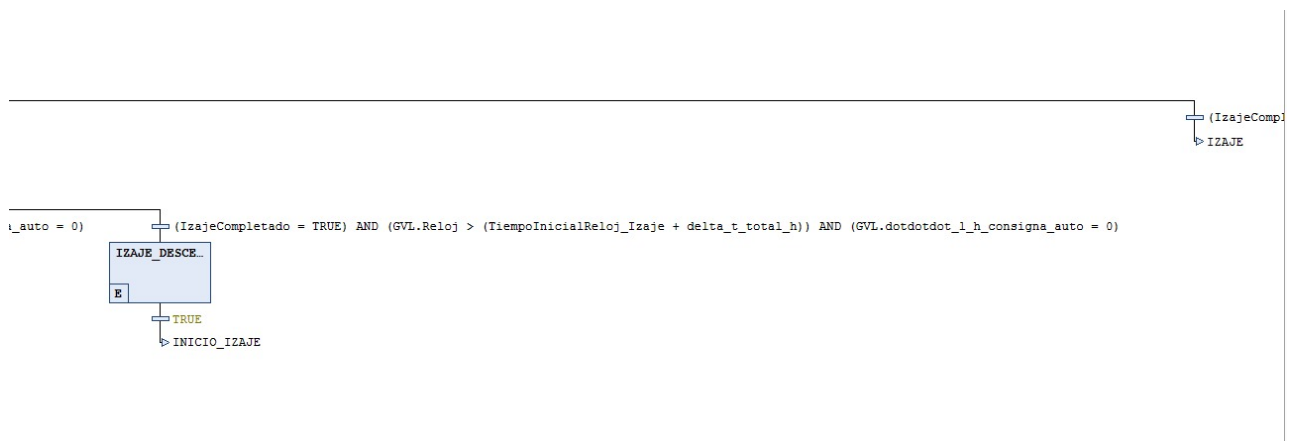


Figura 93: *Control supervisor en SFC CODESYS (13)*

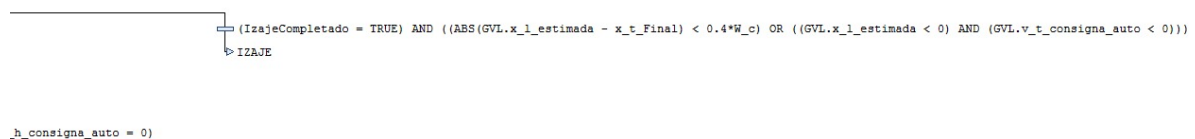


Figura 94: *Control supervisor en SFC CODESYS (14)*

```

1 y_h_consigna_auto := GVL.consignas[2];
2 v_h_consigna_auto := GVL.consignas[3];
3 a_h_consigna_auto := 0;
4 IF (ABS(v_h_consigna_auto) < 0.01 ) THEN
5   v_h_consigna_auto := 0;
6 END IF
7 TiempoInicialReloj_Izaje:=GVL.Reloj;
8 //Trayectoria Vertical;
9 obtenerTiemposPerfilATrapezoidalCalculo(pos_inicial:=y_h_consigna_auto,vel_inicial:=v_h_consigna_auto,acel_inicial:=a_h_consigna_auto,pos_final:=y_h_Final,VelMax:=GVL.v_hMax,i
10

```

Figura 95: IZAJE acción entrada

```

1 moverPorTrayectoriaAutomaticaCalculo(TIEMPOS:=TIEMPOS_h,delta_t_total:=delta_t_total_h,delta_pos_total:=delta_pos_total_h,pos_anterior:=y_h_consigna_auto,vel_anterior:=v_h_c
2 GVL.i_h_consigna_auto := y_t0 - y_h_consigna_auto;
3 GVL.dot_i_h_consigna_auto := -v_h_consigna_auto;
4 GVL.dotdot_i_h_consigna_auto := -a_h_consigna_auto;
5 GVL.dotdotdot_i_h_consigna_auto := -j_h_consigna_auto;
6

```

Figura 96: IZAJE acción activa

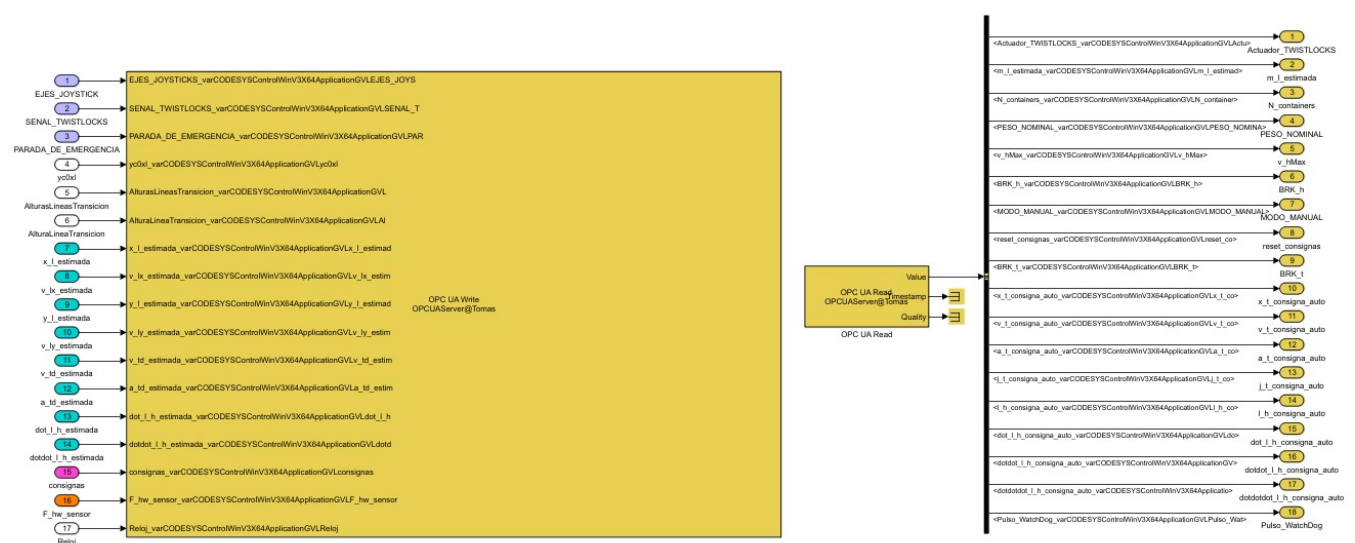


Figura 97: Control supervisor conexión CODESYS con Simulink mediante OPC UA

2.2.3 Implementación del control de seguridad en CODESYS

Al igual que el control supervisor, se desarrolló el control de seguridad en CODESYS para luego ser conectado mediante OPC UA a Simulink (ver apéndice A).

```

[attribute 'qualified_only']
VAR_GLOBAL
    Pulso_WatchDog:LREAL:=0;
    PARADA_DE_EMERGENCIA:BOOL:=FALSE;
    REINICIO:BOOL:=FALSE;
    TiempoMaxWD:LINT:=1000;
    TLSMAX:BOOL:=FALSE;
    TLSMIN:BOOL:=FALSE;
    TLSMAXU:BOOL:=FALSE;
    TLSMINU:BOOL:=FALSE;
    HLSMAX:BOOL:=FALSE;
    HLSMIN:BOOL:=FALSE;
    HLSMAXU:BOOL:=FALSE;
    HLSMINU:BOOL:=FALSE;
    HMAXVD:BOOL:=FALSE;
    EMERGENCIA:LREAL;
    BRK_hE_Nivel0:LREAL;
    BRK_h_Nivel0:LREAL;
    BRK_t_Nivel0:LREAL;
    MODO_MANUAL_E:LREAL;
    reset_consignas_E:LREAL;
END_VAR

```

Figura 98: Definición de las variables de entrada/salida del control de seguridad CODESYS

```

1  PROGRAM PLC_PRG_SEG
2  VAR
3      AWD:BOOL:=FALSE;
4      TemporizadorWD:LINT:=0;
5      PARADA_DE_EMERGENCIA_WD:BOOL:=FALSE;
6      RISING_TLSMAX:R_TRIG;
7      RISING_TLSMIN:R_TRIG;
8      RISING_TLSMAXU:R_TRIG;
9      RISING_TLSMINU:R_TRIG;
10     RISING_HLSMAX:R_TRIG;
11     RISING_HLSMIN:R_TRIG;
12     RISING_HLSMAXU:R_TRIG;
13     RISING_HLSMINU:R_TRIG;
14     RISING_HMAXVD:R_TRIG;
15     RISING_REINICIO:R_TRIG;
16 END_VAR
17

```

Figura 99: Definición de las variables de internas del control de seguridad en CODESYS

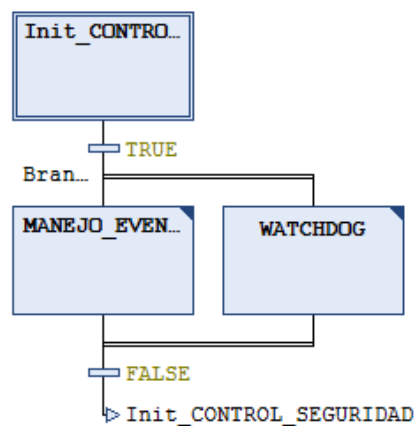


Figura 100: Control de seguridad en CODESYS (1)

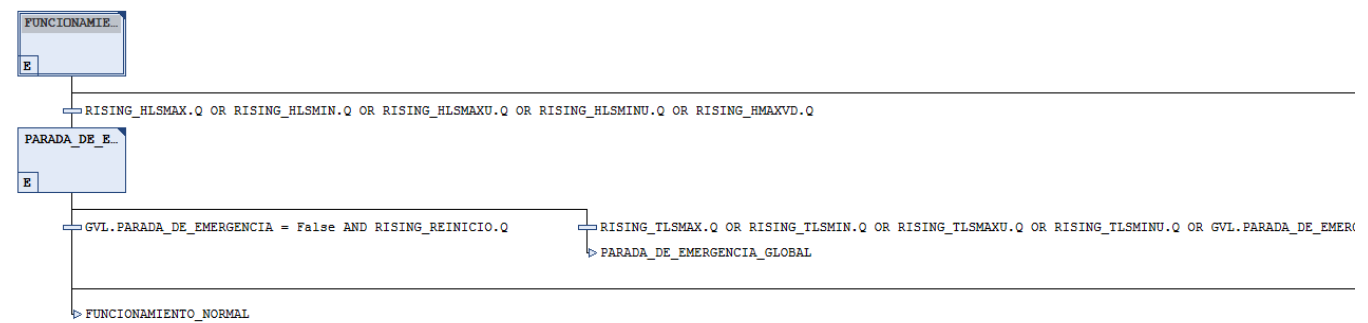


Figura 101: Control de seguridad en CODESYS (2)

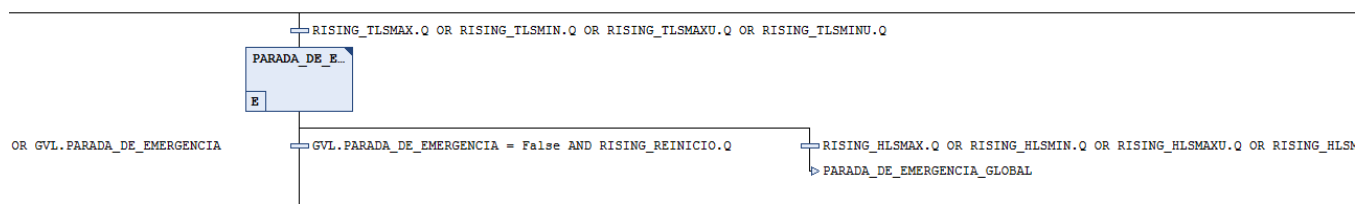


Figura 102: Control de seguridad en CODESYS (3)

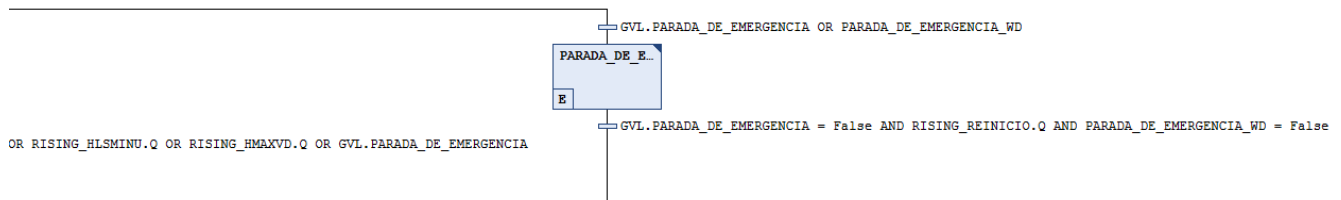


Figura 103: Control de seguridad en CODESYS (4)

En los estados del manejo de emergencias, la acción activa para cada uno resulta ser

```
RISING_TLSMAX (CLK:=GVL.TLSMAX) ;
RISING_TLSMIN (CLK:=GVL.TLSMIN) ;
RISING_TLSMAXU (CLK:=GVL.TLSMAXU) ;
RISING_TLSMINU (CLK:=GVL.TLSMINU) ;
RISING_HLSMAX (CLK:=GVL.HLSMAX) ;
RISING_HLSMIN (CLK:=GVL.HLSMIN) ;
RISING_HLSMAXU (CLK:=GVL.HLSMAXU) ;
RISING_HLSMINU (CLK:=GVL.HLSMINU) ;
RISING_HMAXVD (CLK:=GVL.HMAXVD) ;
RISING_REINICIO (CLK:=GVL.REINICIO) ;
```

Figura 104: Polling flanco de subida en SFC

Mientras que la acción de entrada para el funcionamiento normal es

```
1  GVL.BRK_h_Nivel0 := 1;
2  GVL.BRK_t_Nivel0 := 1;
3  GVL.BRK_hE_Nivel0 := 1;
4  GVL.EMERGENCIA := 0;
5  GVL.reset_consignas_E := 0;
```

Figura 105: *Funcionamiento normal, control de seguridad en SFC*

Para los estado de emergencia es

```
1  GVL.BRK_h_Nivel0 := 0;
2  GVL.BRK_t_Nivel0 := 0;
3  GVL.BRK_hE_Nivel0 := 0;
4  GVL.EMERGENCIA := 1;
5  GVL.MODO_MANUAL_E := 1;
6  GVL.reset_consignas_E := 1;
```

Figura 106: *Estados de emergencia, control de seguridad en SFC*

El funcionamiento del estado de watchdog del control de seguridad en SFC es el siguiente:

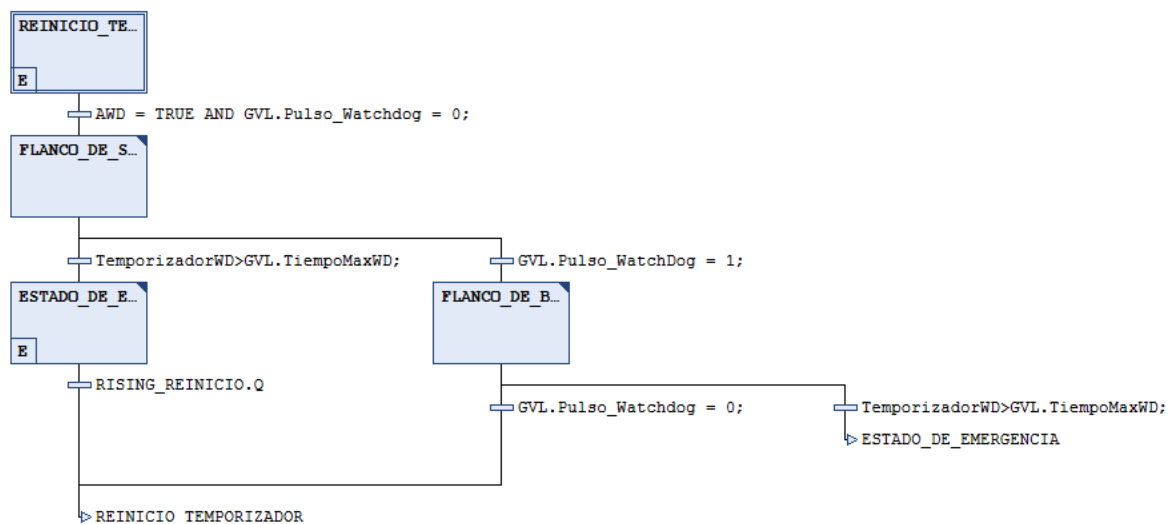


Figura 107: *Estado de WatchDog de control de seguridad en CODESYS*

```
TemporizadorWD:=TemporizadorWD+1;
```

Figura 108: *Acción de suma en estado de WatchDog en CODESYS*

```
PARADA_DE_EMERGENCIA_WD:=True;
```

Figura 109: *Parada de emergencia por detección de WatchDog en CODESYS*

Finalmente se implementa el conexionado mediante OPC UA con Simulink, reemplazando al bloque en StateFlow.



Figura 110: Conexión control de seguridad en CODESYS mediante OPC UA a Simulink

3 CONCLUSIONES

Referencias

- [1] G. L. Julián, "317 AyCD_Proyecto Global Integrador_2024_Guía de TRABAJO_Rev.0.". Jefe de cátedra, Automátas y Control Discreto, Ing. Mecatrónica, Univ. Nac. de Cuyo, 2024.
- [2] H.-H. Lee, "A new approach for the anti-swing control of overhead cranes with high-speed load hoisting". Department of Mechanical Engineering, Tulane University.
- [3] CODESYS Group. "CODESYS OPC UA Server - CODESYS Runtime System". Disponible en: <https://www.codesys.com/products/codesys-runtime/opc-ua.html>.
- [4] MathWorks. "Get Started with OPC UA in Simulink". Disponible en: <https://la.mathworks.com/help/icomm/ug/get-started-opc-ua-simulink.html>.

APÉNDICE A

El apéndice actual tiene por objeto funcionar como un instructivo para conectar el servidor OPC UA del PLC virtual de CODESYS y el cliente de Simulink. A modo de ejemplo, se pretende reemplazar la siguiente máquina de estado finito implementada en Stateflow/Simulink, la cual consiste de básicamente en realizar dos tareas en paralelo, una de parpadeo de un led y otra de actualización de un vector, siempre y cuando el autómata se encuentre en el estado “Estado_Encendido”

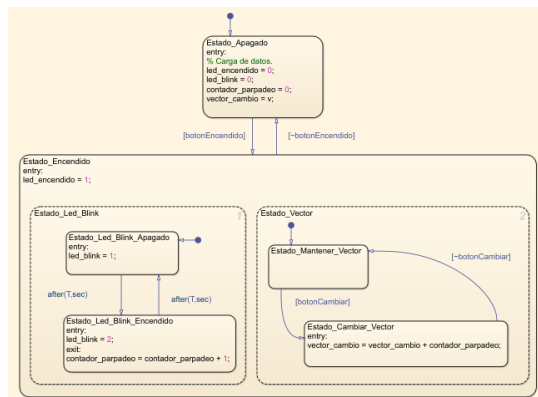


Figura 111: Máquina de estado finito en Stateflow

Una representación análoga puede ser realizada en CODESYS mediante el uso de SFC, generando un programa que corra en la tarea principal del PLC y asignando acciones escritas en el mismo lenguaje a aquellos estados que representan estados con secuencias internas.

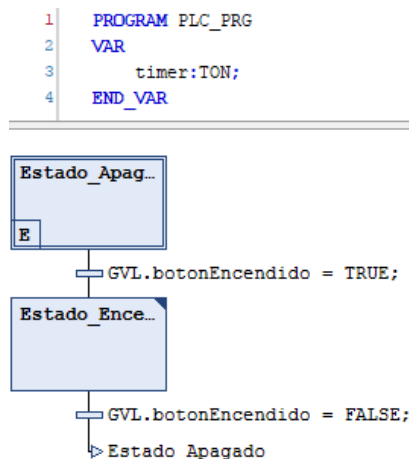


Figura 112: Máquina de estado finito en CODESYS

El autómata posee las siguientes variables de entrada y Salida

Variable	I/O	Descripcion
botonEncendido	I	Botón de encendido
T	I	Semiperíodo de parpadeo
botonCambiar	I	Actualizacion de vector de salida
led_encendido	O	Testigo del estado de encendido
led_blink	O	Led de parpadeo
contador_parpadeo	O	Contador de parpadeos
vector_cambio	O	Vector de salida

Cuadro 1: Entradas y salidas del autómata

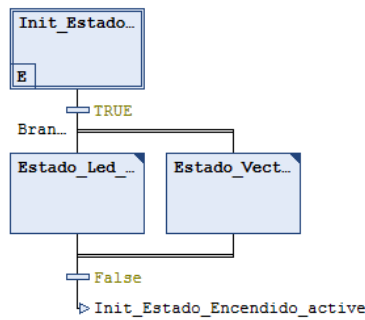


Figura 113: *Estado_Encendido* en CODESYS

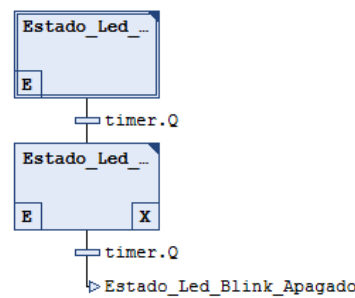


Figura 114: *Estado_Led_Blink* en CODESYS

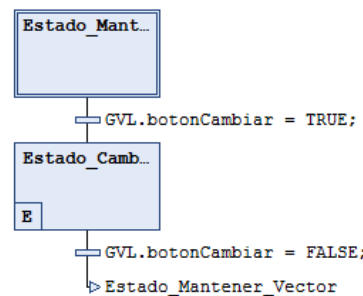


Figura 115: *Estado_Led_Blink* en CODESYS

Las acciones de entrada y salida de los estados resultan ser las mismas que en Stateflow y han sido escritas en *ST*.

A continuación se debe crear un certificado de OPC UA en el servidor para poder ser intercambiado durante la conexión [3]. Para ello, dentro del proyecto, entramos en *Ver/Pantalla de seguridad*, luego en el panel que aparece seleccionamos *Devices/Device/Own Certificates* y luego creamos el certificado.

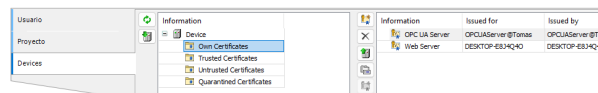
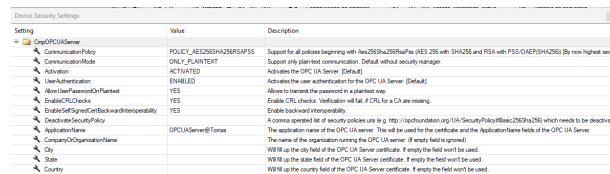


Figura 116: *Certificado OPC UA del servidor* en CODESYS

Las activación y las políticas de seguridad disponibles del servidor OPC UA lanzado en el PLC virtual de codesys deben ser configuradas en *Device/Dispositivo/Configuración de seguridad/CmpOPCUAServer*.



Setting	Value	Description
OpCUAServer		
CommunicationPolicy	POLICY_AES256GCM256RSAPSS	Support for all policies beginning with Aes256Gcm256RsaPss (AES 256 with SHA256 and RSA with PSS (GCM256RSAPSS)) (By now highest security)
CommunicationMode	ONLY_PLAINTEXT	Support only plain text communication. Default without security manager.
Activation	ACTIVATED	Activates the OPC UA Server. [Default]
UserAuthentication	ENABLED	Activates the user authentication for the OPC UA Server. [Default]
AllowUserPasswordOnPort	YES	Allows to forward the password in a plaintext way.
EnableCerts	YES	Enables CRL checker. Verification will fail if CRL for a CA are missing.
EnableSelfSignedCertBackwardCompatibility	YES	Enables backward compatibility.
DeviceSecurityPolicy		A comma separated list of security policies one in g. http://opcfoundation.org/UA/SecurityPolicy#Basic256Sha256 which needs to be deactivated.
ApplicationName	OPCUAServer@Tomas	The application name of the OPC UA server. This will be used for the certificate and the ApplicationName field of the OPC UA Server.
CompanyOrOrganizationName		The name of the organization running the OPC UA server. If empty field is ignored.
City		Will fill up the city field of the OPC UA Server certificate. If empty the field won't be used.
State		Will fill up the state field of the OPC UA Server certificate. If empty the field won't be used.
Country		Will fill up the country field of the OPC UA Server certificate. If empty the field won't be used.

Figura 117: Políticas de seguridad servidor OPC UA en CODESYS

Por el lado de Simulink, pueden utilizarse los bloques *OPC UA Write* y *OPC UA Read*. Esto habilita la posibilidad de conexión a un servidor OPC UA mediante *Model Configuration Parameters*[4]. Mediante el botón *Configure* se puede seleccionar la política de seguridad deseada para la conexión entre las disponibles.

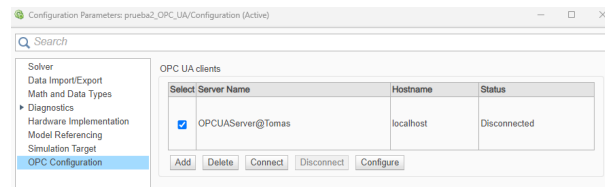


Figura 118: Model Configuration Parameters OPC UA en Simulink

Finalmente mediante los bloques anteriormente mencionados se puede acceder a los nodos del espacio de direcciones del servidor OPC UA para leer o escribir sus variables.